

ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KINH TẾ
KHOA KẾ TOÁN – TÀI CHÍNH



BÀI THI KẾT THÚC HỌC PHẦN

Môn: Tài chính và học máy



SVTH: Nguyễn Khoa Quang Duy

22K4070028

Lớp: K56 CNTC

GVHD: Th.S Hoàng Hữu Trung

Huế, tháng 01/2025

A. THUẬT TOÁN RNN ĐỂ DỰ DOAN GIA	1
I. Giới thiệu về dữ liệu	1
II. Tiền xử lí dữ liệu	1
1. Tìm các giá trị thiếu	1
2. Chia tập dữ liệu thành tập huấn luyện và tập kiểm thử	1
3. Chuẩn hoá dữ liệu bằng phương pháp Min_Max Normalization	2
III. Tiến hành thực hiện thuật toán SimpleRNN	2
1. Xây dựng mô hình	2
2. Kết quả dự đoán của thuật toán SimpleRNN	4
IV. Tiến hành thực hiện thuật toán LSTM để dự đoán	9
1. Xây dựng mô hình LSTM	9
2. Kết quả của thuật toán LSTM	10
V. Thực hiện thuật toán ARIMA để dự đoán	15
1. Xây dựng thuật toán ARIMA	15
2. Thực hiện thuật toán	18
VI. Đánh giá thuật toán	23
B. THUẬT TOÁN CNN ĐỂ PHAN LOẠI ẢNH	26
I. Giới thiệu bộ dữ liệu	26
II. Tiền xử lí dữ liệu	27
1. Nhập thư viện	27
2. Tiến hành giảm số lớp	28
3. Đếm số ảnh của mỗi lớp	28
4. Chuẩn hoá dữ liệu với phương pháp Min_Max_Normalization	29
5. Hiện ảnh ngẫu nhiên với mỗi lớp hoạt động	29
III. Thực hiện thuật toán Alexnet với bộ dữ liệu	30
2. Tiến hành thuật toán phân loại hoạt động với mô hình Alexnet	31
3. Kết quả của thuật toán	33
IV. Sử dụng thuật toán Mobilenet để phân loại	35
1. Tạo ra mạng Mobilenet	35
2. Tiến hành thuật toán phân loại hoạt động với mô hình Mobilenet	36

3.	Kết quả thuật toán	42
V.	<i>Ứng dụng thuật toán Resnet để phân loại hoạt động</i>	44
1.	Tạo ra mạng Resnet	44
2.	Tiến hành thuật toán Resnet	45
VI.	<i>Đánh giá, so sánh kết quả phân loại của các thuật toán</i>	49

A. Thuật toán RNN để dự đoán giá

I. Giới thiệu về dữ liệu

Bộ dữ liệu gồm giá của 2 loại đồng tiền điện tử là Binance và Wrapped Bitcoin, cùng với 3 mã cổ phiếu của 3 doanh nghiệp có vốn hoá thị trường đầu trên thế giới là Tesla, Ford và Intel Corp.

STT	Kí hiệu	Chú thích
1	BNB-USD	Đồng tiền điện tử Binance
2	WBTC-USD	Đồng tiền điện tử Wrapped Bitcoin
3	TSLA	Mã cổ phiếu Tesla
4	INTC	Mã cổ phiếu Intel Corp
5	F	Mã cổ phiếu Ford

Trong bối cảnh thị trường tài chính ngày càng phức tạp, việc dự đoán giá của các tài sản như tiền điện tử và cổ phiếu trở thành một lĩnh vực nghiên cứu quan trọng. Bài viết này tập trung vào việc xây dựng và áp dụng các thuật toán dự đoán giá cho hai đồng tiền điện tử phổ biến là BNB (Binance Coin) và WBTC (Wrapped Bitcoin), cùng ba mã cổ phiếu nổi bật trên thị trường là TSLA (Tesla, Inc.), F (Ford Motor Company) và INTC (Intel Corporation). Thông qua việc kết hợp các phương pháp phân tích dữ liệu lịch sử, học máy (machine learning) và trí tuệ nhân tạo, nghiên cứu này nhằm mang đến dự đoán chính xác hơn, hỗ trợ nhà đầu tư ra quyết định hiệu quả trong môi trường biến động không ngừng của thị trường.

Thuật toán sử dụng giá của những đồng tiền điện tử và cổ phiếu trên từ 01/01/2019 đến 31/12/2024 để tiến hành xây dựng thuật toán và đánh giá.

II. Tiền xử lí dữ liệu

1. Tìm các giá trị thiếu

Kiểm tra xem dữ liệu có bị thiếu hay không.

```
data.isnull()
```

➔ Không có dữ liệu thiếu

2. Chia tập dữ liệu thành tập huấn luyện và tập kiểm thử

```
def create_dataset(dataset, time_step=365):
```

```
X, y = [], []
for i in range(len(dataset) - time_step):
    X.append(dataset[i:i + time_step, 0])
    y.append(dataset[i + time_step, 0])
return np.array(X), np.array(y)
```

Tiến hành tạo ra dataset với số bước là 365.

Chia bộ dữ liệu thành tập huấn luyện và tập kiểm thử với tỉ lệ là 8/2.

Lấy năm 2024 làm tập dữ liệu kiểm tra.

3. Chuẩn hoá dữ liệu bằng phương pháp Min_Max Normalization

```
time_step = 365
train_size = int(len(bnb_scaled) * 0.8)
X_train, y_train = create_dataset(bnb_scaled[:train_size], time_step)
X_test, y_test = create_dataset(bnb_scaled[train_size - time_step:], time_step)
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
```

Khai báo số bước là 365.

Chuẩn hoá dữ liệu về giá trị [0 ;1].

III. Tiến hành thực hiện thuật toán SimpleRNN

1. Xây dựng mô hình

```
model = Sequential()
model.add(SimpleRNN(50, return_sequences=True, input_shape=(time_step, 1)))
model.add(SimpleRNN(50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=1)
```

Xây dựng mô hình SimpleRNN với:

- Mô hình có 50 đơn vị neuron.
- Mô hình có đầu vào có 1 đặc trưng và có số giá trị bằng time_step.
- Ở đây tiến hành cho học tập dữ liệu với 10 lần (epochs=10).

Sử dụng RMSE để đánh giá kết quả dự đoán của mô hình.

```
train_pred = scaler.inverse_transform(model.predict(X_train))
```

```
test_pred = scaler.inverse_transform(model.predict(X_test))
y_train_actual = scaler.inverse_transform(y_train.reshape(-1, 1))
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))
train_rmse = np.sqrt(mean_squared_error(y_train_actual, train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test_actual, test_pred))
print(f'Train RMSE: {train_rmse}')
print(f'Test RMSE: {test_rmse}')
```

Cho tiến hành dự đoán ở tập huấn luyện và tập kiểm tra.

Tính toán RMSE ở cả tập huấn luyện và tập kiểm thử để kiểm tra kết quả thu về.

```
future_days = 366
future_predictions = []
last_days = bnb_scaled[-time_step:]
for _ in range(future_days):
    pred = model.predict(last_days.reshape(1, -1, 1))[0, 0]
    future_predictions.append(pred)
    last_days = np.append(last_days[1:], pred)
```

Tiến hành dự đoán. Ở đây cho biến future_days = 366 vì năm 2024 là năm nhuận.

Thêm kết quả liên tục với mỗi dự đoán cho đến ngày 31/12/2024.

```
future_predictions =
scaler.inverse_transform(np.array(future_predictions).reshape(-1, 1))
future_dates = [datetime(2024, 1, 1) + timedelta(days=i) for i in
range(future_days)]
comparison_df = pd.DataFrame({
    'Date': bnb.index[-len(test_pred):],
    'Actual': y_test_actual.flatten(),
    'Predicted': test_pred.flatten()})
print(comparison_df.head(10))
```

Tiến hành đảo ngược chuẩn hoá, đưa giá cổ phiếu về lại ban đầu.

Tiến hành so sánh giữa giá thực tế và giá dự đoán.

Hiện 10 kết quả đầu tiên.

```
bnb1= yf.download('BNB-USD', start='2024-01-01', end='2024-12-31')['Close']
```

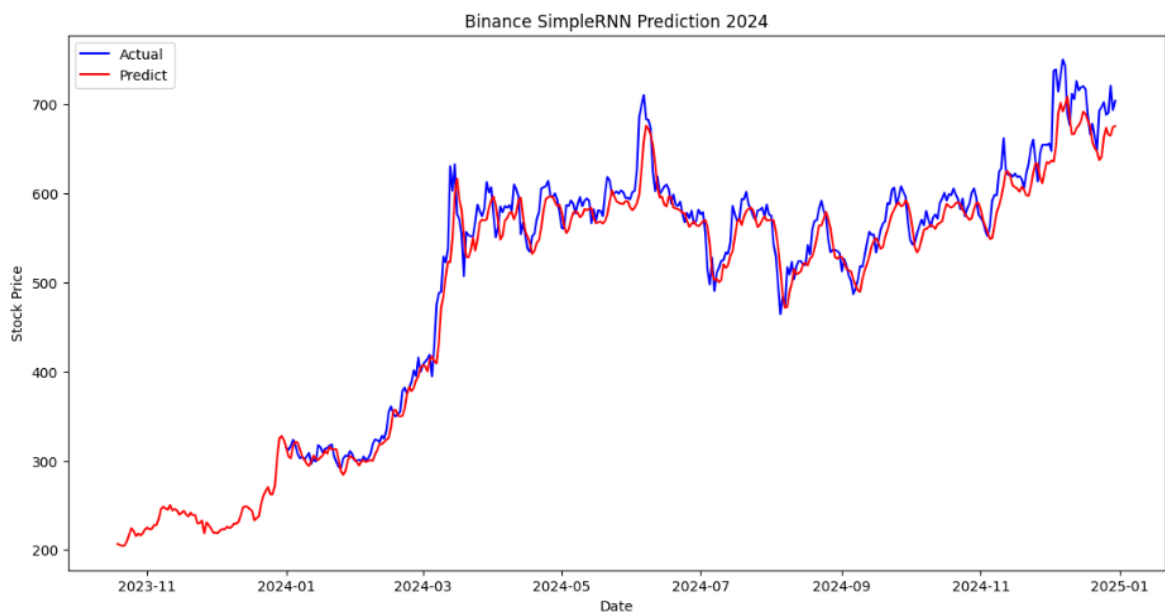
```
plt.figure(figsize=(14, 7))
plt.plot(bnb1.index, bnb1, label='Actual', color='blue')
plt.plot(bnb.index[-len(test_pred):], test_pred, label='Predict', color='red')
plt.title('Binance SimpleRNN Prediction 2024')
plt.xlabel('Date')
plt.ylabel('Stock Price')
plt.legend()
plt.show()
```

Tiến hành trực quan hoá dữ liệu lên trên biểu đồ

- Đường màu xanh là giá thực tế
- Đường màu đỏ là giá dự đoán

2. Kết quả dự đoán của thuật toán SimpleRNN

a. Kết quả dự đoán của BNB-USD



Hình 1: Kết quả dự đoán của BNB-USD

Train RMSE : 17.509109262116926

Test RMSE: 22.19363599713923

0	2023-10-19	211.144165	206.634644
1	2023-10-20	212.300842	205.385300
2	2023-10-21	214.305664	204.532043
3	2023-10-22	217.747375	204.830353
4	2023-10-23	228.352768	209.349915
5	2023-10-24	225.142960	217.111008
6	2023-10-25	222.179932	224.242599
7	2023-10-26	223.236038	221.089188
8	2023-10-27	224.475906	215.371994
9	2023-10-28	225.774078	218.125656

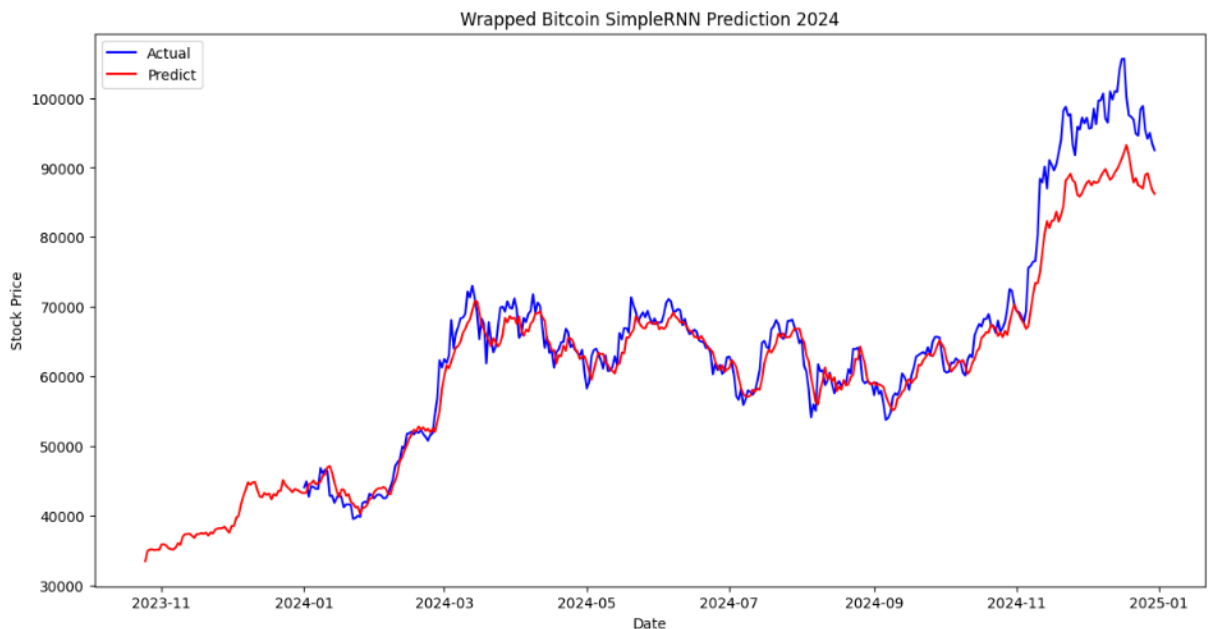
➔ Kết quả đánh giá bằng RMSE cho thấy giá đồng BNB có sự chênh lệch không quá lớn (Khoảng 22.19). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán khá sát với giá trị thực tế (Khoảng cách lớn nhất là 17USD ở ngày 23/10/2023). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày. Vì thế các nhà đầu tư nên có sự chuẩn bị sớm để thực hiện các hoạt động đầu tư.

b. Kết quả dự đoán của WBTC-USD

Train RMSE: 1551.2335777284554

Test RMSE: 3739.5851903153034

Đồ thị giữa giá thực tế và giá dự đoán WBTC-USD:



Kết quả dự đoán của WBTC-USD

	Date	Actual	Predicted
0	2023-10-25	34496.593750	33468.316406
1	2023-10-26	34187.242188	34935.636719
2	2023-10-27	33865.847656	35160.152344
3	2023-10-28	34080.859375	35186.445312
4	2023-10-29	34572.492188	35053.285156
5	2023-10-30	34481.378906	35154.746094
6	2023-10-31	34595.910156	35105.371094
7	2023-11-01	35353.406250	35895.925781
8	2023-11-02	34843.050781	35905.824219
9	2023-11-03	34706.644531	35717.007812

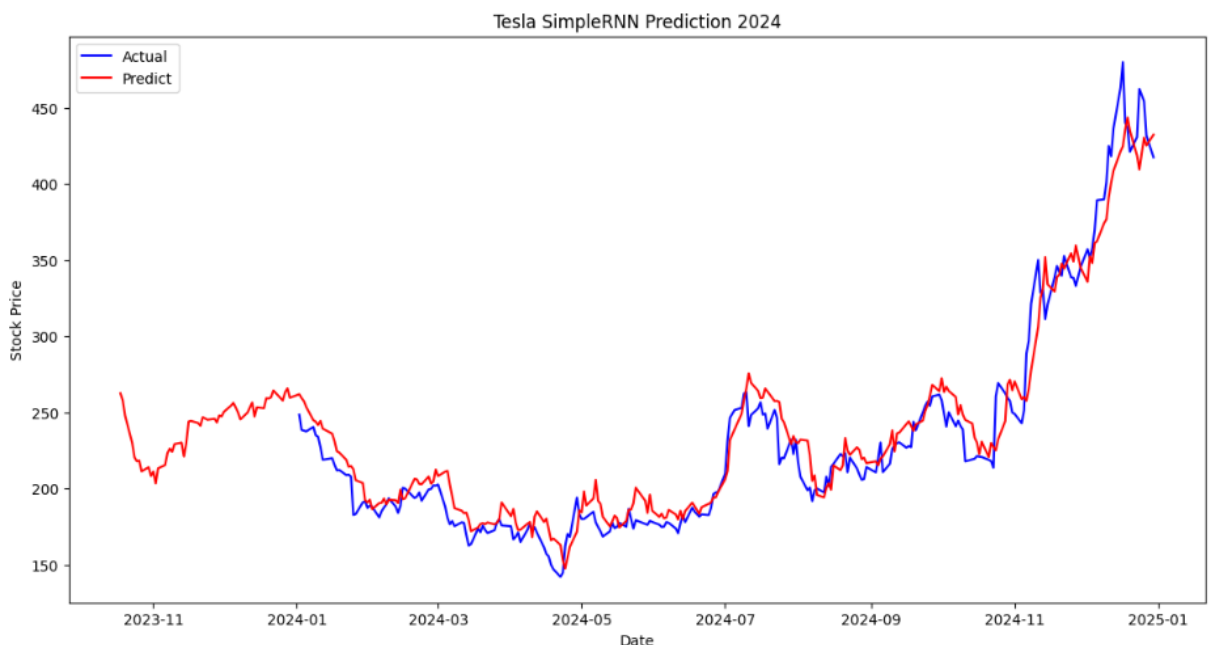
➔ Kết quả đánh giá bằng RMSE cho thấy giá đồng WBTC có sự chênh lệch trông có vẻ lớn (Khoảng 3739.58USD), song với mức giá hiện nay lên đến 100.000USD thì sự chênh lệch trên lại tương đối nhỏ. Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán khá sát với giá trị thực tế (Khoảng từ 1000 đến 1500 USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời cũng đôi lúc đánh giá thấp tiềm năng của đồng trên. Vì thế các nhà đầu tư nên có sự chuẩn bị sớm để thực hiện các hoạt động đầu tư.

c. Kết quả của cổ phiếu TSLA

Train RMSE: 13.819353248195544

Test RMSE: 15.095396929602066

Đồ thị giữa giá thực tế và giá dự đoán cổ phiếu TSLA:



Kết quả dự đoán của TSLA

	Date	Actual	Predicted
0	2023-10-18	242.679993	262.490479
1	2023-10-19	220.110001	257.773163
2	2023-10-20	211.990005	247.811707
3	2023-10-23	212.080002	229.996185
4	2023-10-24	216.520004	220.460144
5	2023-10-25	212.419998	217.885849
6	2023-10-26	205.759995	218.412842
7	2023-10-27	207.300003	211.282928
8	2023-10-30	197.360001	213.996277
9	2023-10-31	200.839996	208.263046

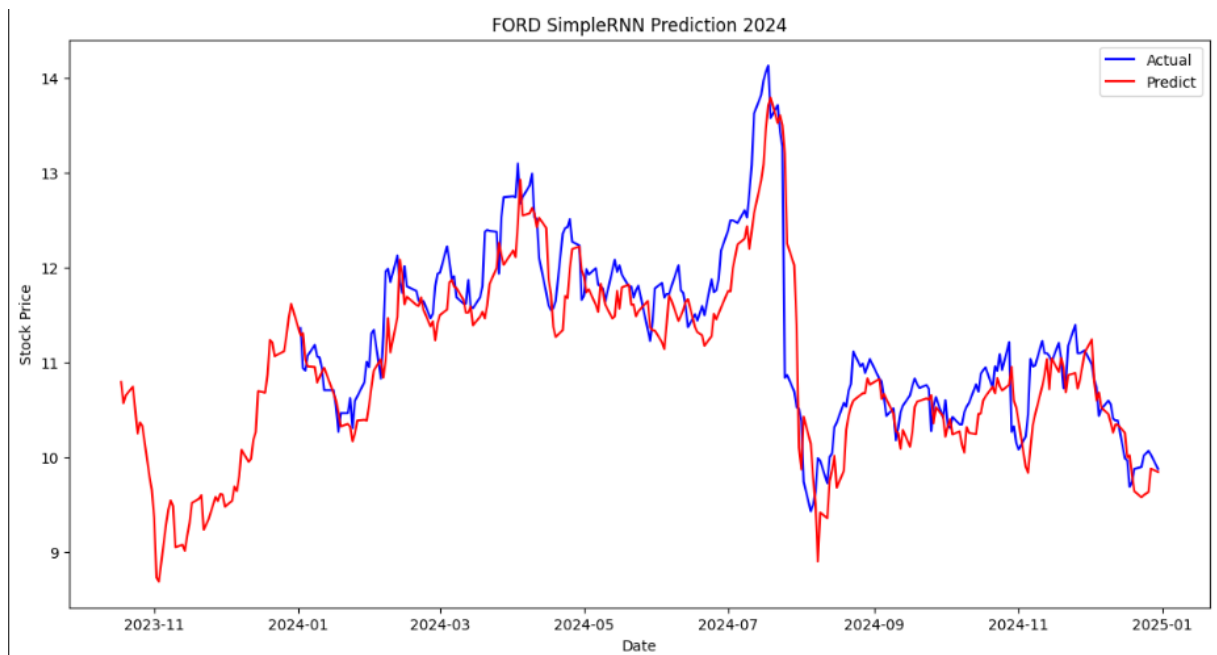
➔ Kết quả đánh giá bằng RMSE cho thấy giá cổ phiếu TSLA có sự chênh lệch thấp (Khoảng 15.09). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán khá sát với giá trị thực tế (Khoảng cách lớn nhất không quá 20USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời giá cổ phiếu đang được đánh giá qua cao so với thực tiễn. Vì thế các nhà đầu tư nên có sự xem xét để thực hiện các hoạt động đầu tư.

d. Kết quả của cổ phiếu F

Train RMSE: 0.48748571283585623

Test RMSE: 0.43218882869828473

Đồ thị giữa giá thực tế và giá dự đoán cổ phiếu F:



Hình 4: Kết quả dự đoán giá cổ phiếu F

	Date	Actual	Predicted
0	2023-10-18	10.812809	10.795919
1	2023-10-19	10.610355	10.572651
2	2023-10-20	10.711583	10.655993
3	2023-10-23	10.564343	10.747260
4	2023-10-24	10.481524	10.497516
5	2023-10-25	10.619559	10.251889
6	2023-10-26	10.444713	10.369095
7	2023-10-27	9.165581	10.331893
8	2023-10-30	8.990736	9.804977
9	2023-10-31	9.112231	9.658287

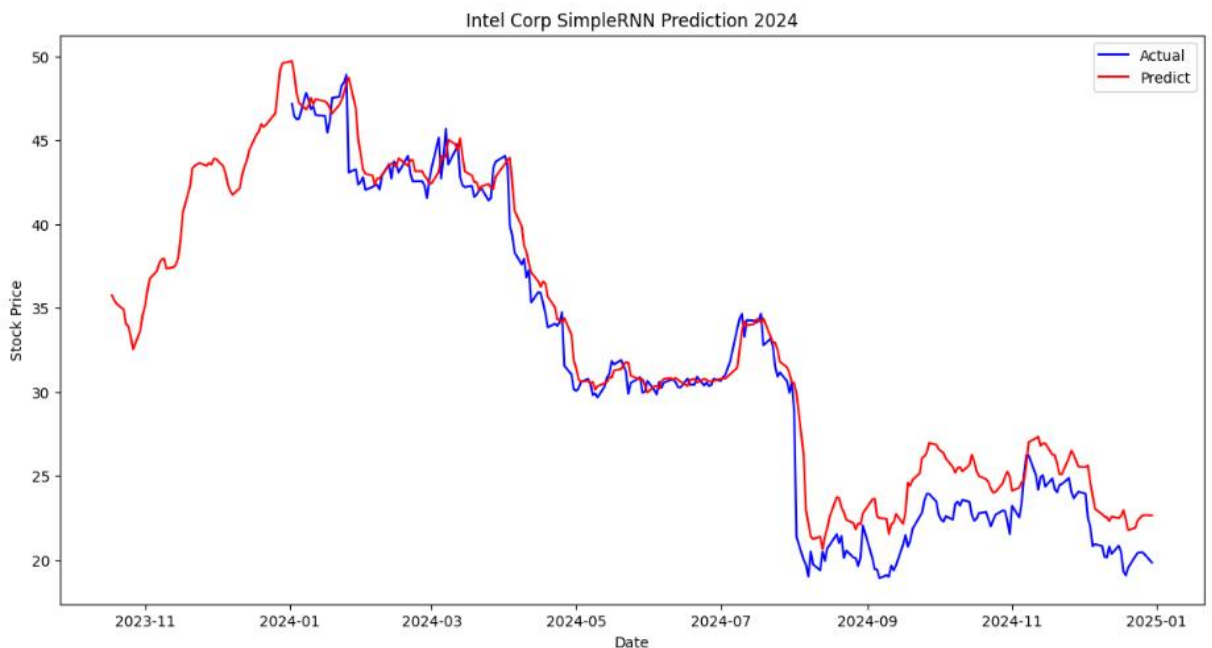
➔ Kết quả đánh giá bằng RMSE cho thấy giá cổ phiếu F có sự chênh lệch rất thấp (Khoảng 0.43USD). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán sát với giá trị thực tế (Khoảng cách lớn nhất không quá 1USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời cảm giác khoảng cách cũng khá xa bởi vì giá cổ phiếu F thấp so với những cổ phiếu còn lại. Vì thế các nhà đầu tư nên có sự xem xét để thực hiện các hoạt động đầu tư.

e. Kết quả của cổ phiếu INTC

Train RMSE: 1.199640602586318

Test RMSE: 1.8520371607943469

Đồ thị biểu diễn giá thực tế và giá dự đoán cổ phiếu INTC:



Hình 5: Kết quả dự đoán giá cổ phiếu INTC

	Date	Actual	Predicted
0	2023-10-18	35.053722	35.754971
1	2023-10-19	35.083229	35.460464
2	2023-10-20	34.345570	35.251083
3	2023-10-23	33.293171	34.895176
4	2023-10-24	34.021000	34.036831
5	2023-10-25	32.289948	33.890720
6	2023-10-26	31.985048	33.263515
7	2023-10-27	34.955364	32.522430
8	2023-10-30	35.102901	33.656441
9	2023-10-31	35.899574	34.590691

➔ Kết quả đánh giá bằng RMSE cho thấy giá cổ phiếu INTC có sự chênh lệch thấp (Khoảng 1.20USD). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán sát với giá trị thực tế (Khoảng cách lớn nhất không quá 2USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời thuật toán cũng đang đánh giá giá trị của cổ phiếu cao hơn so với thực tế. Vì thế các nhà đầu tư nên có sự xem xét để thực hiện các hoạt động đầu tư.

IV. Tiến hành thực hiện thuật toán LSTM để dự đoán

1. Xây dựng mô hình LSTM

```
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(time_step, 1)))
model.add(LSTM(50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=1)
```

Xây dựng mô hình LSTM

- Mô hình có 50 đơn vị neuron
- Mô hình có đầu vào có 1 đặc trưng và có số giá trị bằng time_step
- Ở đây tiến hành cho học tập dữ liệu với 10 lần (epochs=10)

Sử dụng RMSE để làm kết quả hao hụt

```
future_days = 365
future_predictions = []
last_60_days = data_scaled[-time_step:]
for _ in range(future_days):
    pred = model.predict(last_60_days.reshape(1, -1, 1))[0, 0]
```

```
future_predictions.append(pred)
last_60_days = np.append(last_60_days[1:],
```

Tiến hành dự đoán

Thêm kết quả liên tục với mỗi dự đoán cho đến ngày 31/12/2024

```
future_predictions =
scaler.inverse_transform(np.array(future_predictions).reshape(-1, 1))
future_dates = [datetime(2024, 1, 1) + timedelta(days=i) for i in
range(future_days)]
comparison_df = pd.DataFrame({
    'Date': data.index[-len(test_pred):],
    'Actual': y_test_actual.flatten(),
    'Predicted': test_pred.flatten()})
print(comparison_df.head(10))
```

Tiến hành đảo ngược chuẩn hoá, đưa giá cổ phiếu về lại ban đầu

Tiến hành so sánh giữa giá thực tế và giá dự đoán

Hiện 10 kết quả đầu tiên

Tiến hành trực quan hoá dữ liệu

Hiện kết quả dự đoán giá trong năm 2024

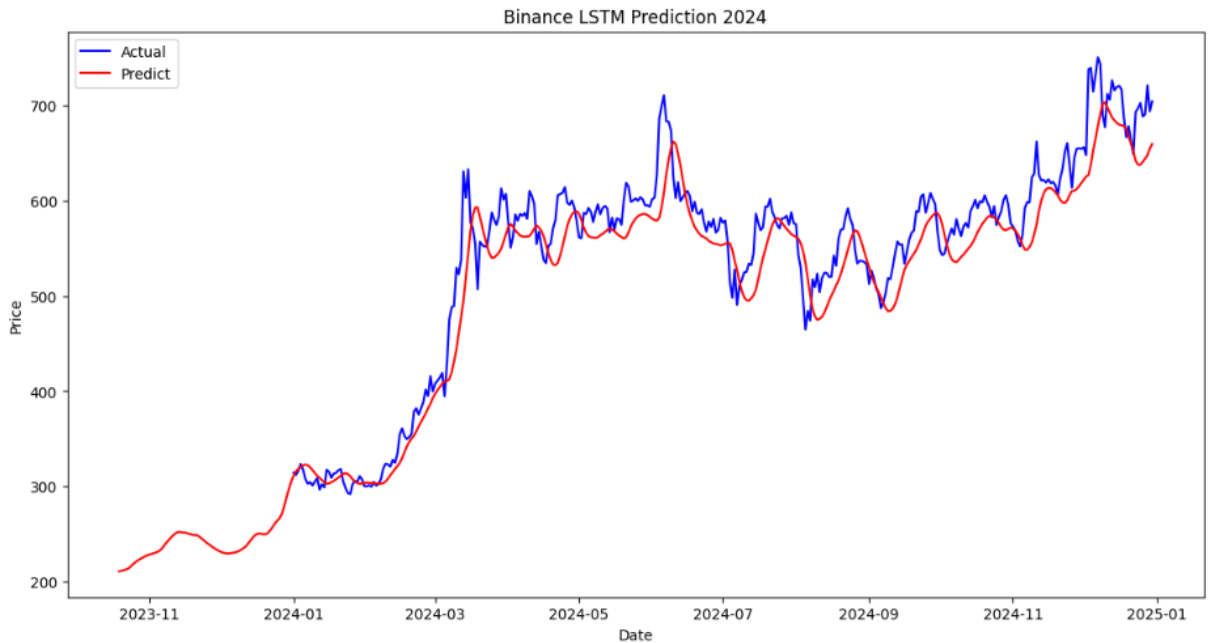
```
plt.figure(figsize=(14, 7))
data1= yf.download('BNB-USD', start='2024-01-01', end='2024-12-31')['Close']
plt.plot(data1.index, data1, label='Actual', color='blue')
plt.plot(data.index[-len(test_pred):], test_pred, label='Predict', color='red')
plt.title('Binance LSTM Prediction 2024')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()
```

2. Kết quả của thuật toán LSTM

a. Kết quả dự đoán giá BNB-USD

Train RMSE: 23.860467363249604

Test RMSE: 31.519740786734562



Kết quả dự đoán giá BNB-USD

0	2023-10-19	211.144165	210.839767
1	2023-10-20	212.300842	211.450104
2	2023-10-21	214.305664	212.105606
3	2023-10-22	217.747375	212.905029
4	2023-10-23	228.352768	214.026077
5	2023-10-24	225.142960	216.176651
6	2023-10-25	222.179932	218.539307
7	2023-10-26	223.236038	220.610413
8	2023-10-27	224.475906	222.408478
9	2023-10-28	225.774078	223.970444

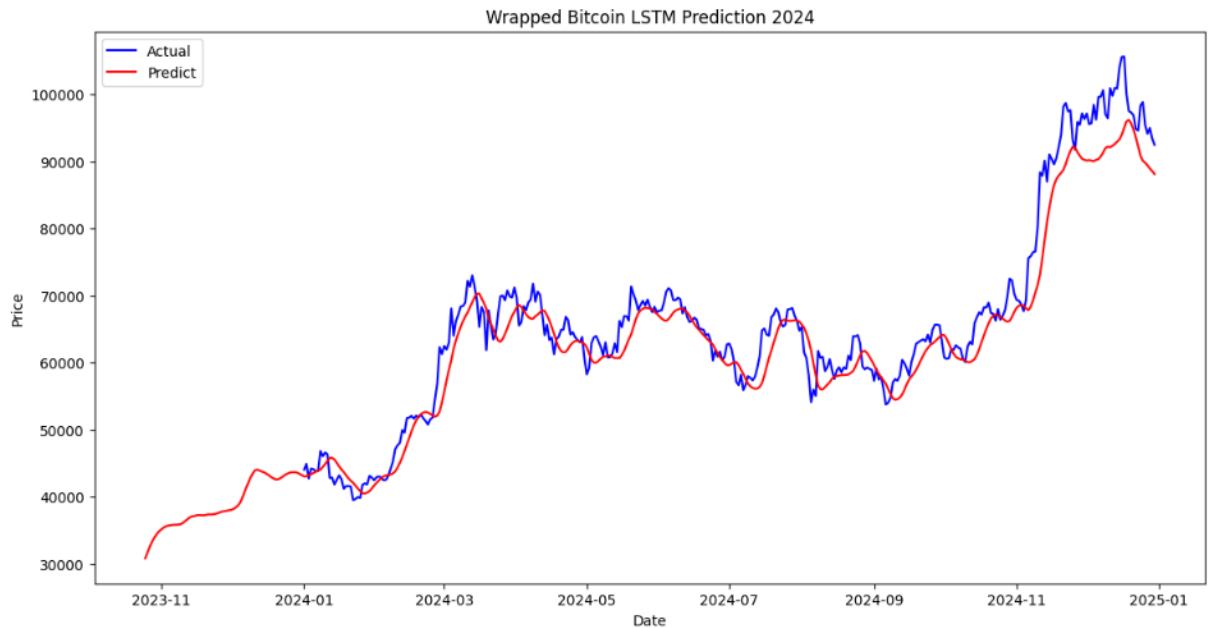
➔ Kết quả đánh giá bằng RMSE cho thấy giá đồng BNB có sự chênh lệch tạm chấp nhận được (Khoảng 31.52 USD). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán khá sát với giá trị thực tế (Không quá 10 USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình chỉ cho thấy xu hướng của giá chứ không dự đoán những sự thay đổi nhỏ, đồng thời dự đoán có gặp vấn đề khi dự đoán chậm trong khoảng 2 – 3 ngày. Vì thế các nhà đầu tư nên có sự chuẩn bị sớm để thực hiện các hoạt động đầu tư.

b. Kết quả dự đoán giá WBTC-USD

Train RMSE: 2051.2753455019833

Test RMSE: 3563.0114904672355

Đồ thị giữa giá thực tế và giá dự đoán WBTC-USD:



Kết quả dự đoán giá WBTC-USD

0	2023-10-25	34496.593750	30853.777344
1	2023-10-26	34187.242188	31761.363281
2	2023-10-27	33865.847656	32631.703125
3	2023-10-28	34080.859375	33373.417969
4	2023-10-29	34572.492188	33985.566406
5	2023-10-30	34481.378906	34502.343750
6	2023-10-31	34595.910156	34899.851562
7	2023-11-01	35353.406250	35194.664062
8	2023-11-02	34843.050781	35465.789062
9	2023-11-03	34706.644531	35645.855469

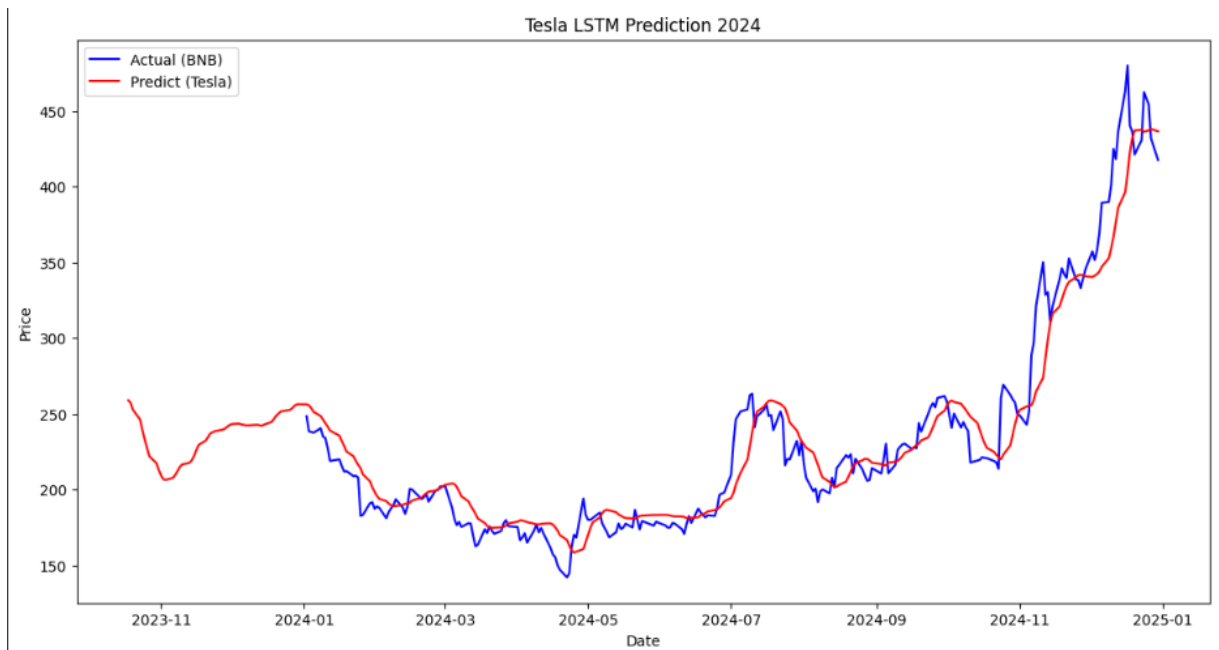
➔ Kết quả đánh giá bằng RMSE cho thấy giá đồng WBTC có sự chênh lệch trông có vẻ lớn (Khoảng 3563.01 USD), song với mức giá hiện nay lên đến 100.000 USD thì sự chênh lệch trên lại tương đối nhỏ. Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán khá sát với giá trị thực tế (Khoảng lớn nhất là 4000 USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời cũng đôi lúc đánh giá thấp tiềm năng của đồng trên.

c. Kết quả dự đoán giá TSLA

Train RMSE: 15.682920184018407

Test RMSE: 17.598063317607238

Đồ thị giữa giá thực tế và giá dự đoán TSLA:



Hình 8: Kết quả dự đoán giá TSLA

0	2023-10-18	242.679993	258.898254
1	2023-10-19	220.110001	257.083496
2	2023-10-20	211.990005	252.592758
3	2023-10-23	212.080002	246.203720
4	2023-10-24	216.520004	239.251099
5	2023-10-25	212.419998	232.975662
6	2023-10-26	205.759995	227.288605
7	2023-10-27	207.300003	221.899200
8	2023-10-30	197.360001	217.359146
9	2023-10-31	200.839996	212.854385

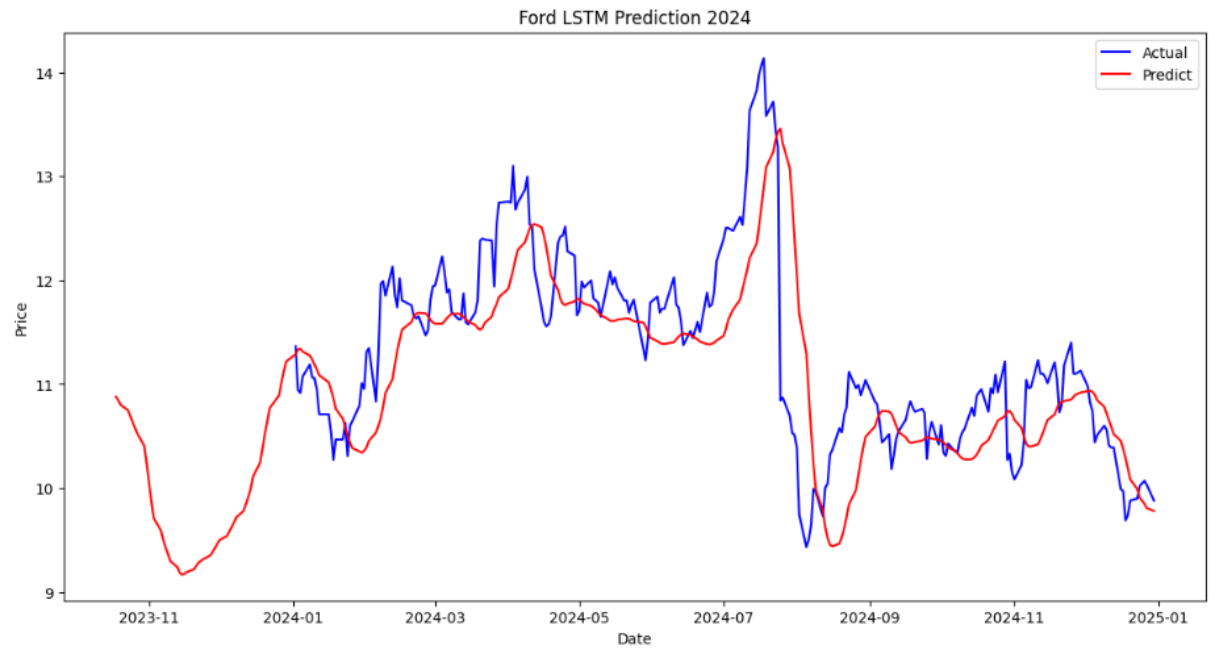
➔ Kết quả đánh giá bằng RMSE cho thấy giá cổ phiếu TSLA có sự chênh lệch thấp (Khoảng 17.59). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán khá sát với giá trị thực tế (Khoảng cách lớn nhất không quá 20 USD). Đồng thời đồ thị cũng đem đến kết quả khả quan khá tương đồng khi sự cách biệt giữa giá thực tế và giá dự đoán khá sát nhau. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời giá cổ phiếu đang được đánh giá tương đối sát với thực tiễn.

d. Kết quả dự đoán giá F

Train RMSE: 0.6840314911733606

Test RMSE: 0.6375324448680534

Đồ thị giữa giá thực tế và giá dự đoán F:



Hình 9: Kết quả dự đoán giá cổ phiếu F

	Date	Actual	Predicted
0	2023-10-18	10.812809	10.878007
1	2023-10-19	10.610355	10.845583
2	2023-10-20	10.711583	10.799666
3	2023-10-23	10.564343	10.752423
4	2023-10-24	10.481524	10.698125
5	2023-10-25	10.619559	10.637755
6	2023-10-26	10.444713	10.583712
7	2023-10-27	9.165581	10.527130
8	2023-10-30	8.990736	10.404899
9	2023-10-31	9.112231	10.237528

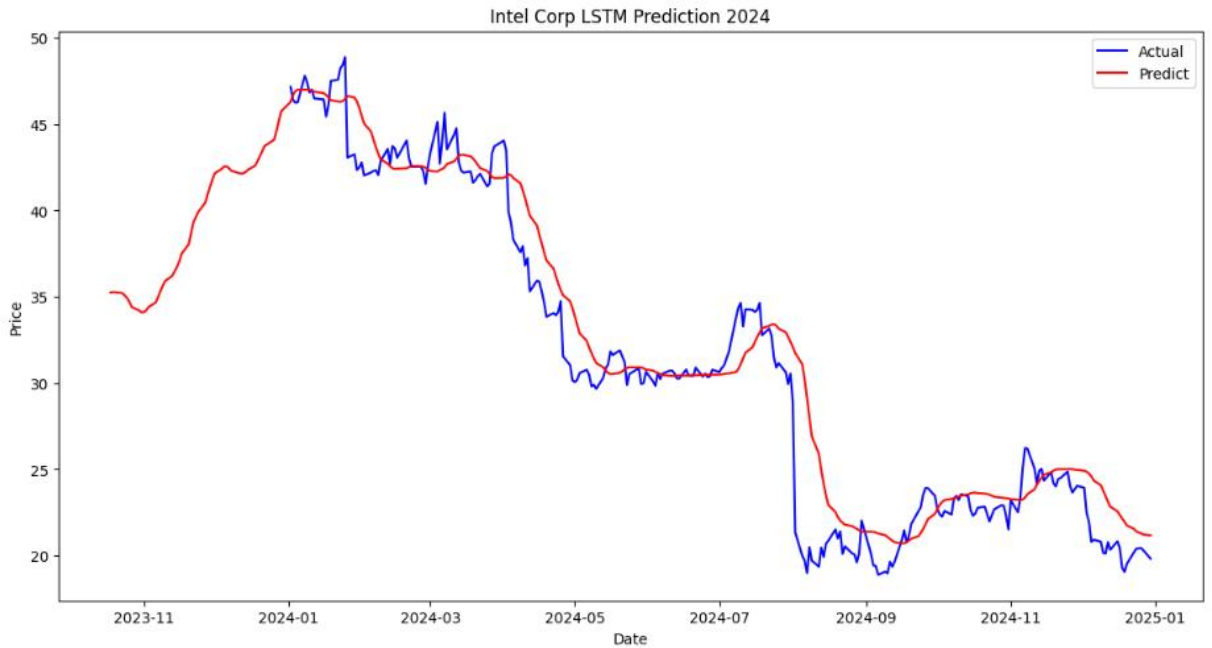
➔ Kết quả đánh giá bằng RMSE cho thấy giá cổ phiếu F có sự chênh lệch rất thấp (Khoảng 0.63 USD). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán sát với giá trị thực tế (Khoảng cách lớn nhất không quá 1 USD). Song nhìn trên đồ thị lại cho thấy cổ phiếu dự đoán từ tháng 1 đến tháng 6 có sự sai lệch tương đối về xu hướng, điều này có thể xuất phát từ việc giá của cổ phiếu trên không quá cao (Cao nhất là 14 USD), từ đó có thể dẫn đến sự sai sót. Bên cạnh đó giá cổ phiếu cũn được dự đoán tương đối chậm (từ 2 đến 3 ngày). Vì thế các nhà đầu tư nên xem xét đường xu hướng để có quyết định đầu tư phù hợp.

e. Kết quả dự đoán giá INTC

Train RMSE: 1.9413180701225927

Test RMSE: 2.467489063377872

Đồ thị giữa giá thực tế và giá dự đoán INTC:



Hình 10: Kết quả dự đoán giá cổ phiếu INTC

0	2023-10-18	35.053722	35.247616
1	2023-10-19	35.083229	35.272636
2	2023-10-20	34.345570	35.268665
3	2023-10-23	33.293171	35.212421
4	2023-10-24	34.021000	35.077541
5	2023-10-25	32.289948	34.925117
6	2023-10-26	31.985048	34.692757
7	2023-10-27	34.955364	34.404301
8	2023-10-30	35.102901	34.214779
9	2023-10-31	35.899574	34.109581

➔ Kết quả đánh giá bằng RMSE cho thấy giá cổ phiếu INTC có sự chênh lệch thấp (Khoảng 2.46 USD). Bên cạnh đó dựa trên 10 kết quả đầu tiên cũng có dự đoán sát với giá trị thực tế (Khoảng cách lớn nhất không quá 2USD). Đồ thị cho ta thấy rõ đường xu hướng đi xuống của cổ phiếu. Song mô hình vẫn có chút vấn đề khi vẫn dự đoán chậm từ khoảng 2 đến 3 ngày, đồng thời thuật toán cũng đang đánh giá giá trị của cổ phiếu cao hơn so với thực tế. Vì thế các nhà đầu tư nên có sự xem xét để thực hiện các hoạt động đầu tư.

V. Thực hiện thuật toán ARIMA để dự đoán

1. Xây dựng thuật toán ARIMA

```
# Test for stationarity with thinner lines and smaller fonts
def test_stationarity(timeseries):
    # Determining rolling statistics
    rolmean = timeseries.rolling(12).mean()
    rolstd = timeseries.rolling(12).std()
```

```
# Plot rolling statistics with thinner lines
plt.figure(figsize=(10, 6)) # Adjust figure size if needed
plt.plot(timeseries, color='blue', label='Original', linewidth=0.8) # Thin blue line
plt.plot(rolmean, color='red', label='Rolling Mean', linewidth=0.8) # Thin red line
plt.plot(rolstd, color='black', label='Rolling Std', linewidth=0.8) # Thin black line
plt.legend(loc='best', fontsize=10) # Smaller font for legend
plt.title('Rolling Mean and Standard Deviation', fontsize=12) # Smaller title
plt.show(block=False)
```

Xác định và vẽ đồ thị thống kê trượt

In kết quả kiểm định Dickey-Fuller

Chuyển kết quả kiểm định thành một pandas.Series để dễ đọc

Xu hướng (trend): Giúp quan sát thấy xu hướng của cổ phiếu

Tính mùa vụ (Seasonal) : Giúp quan sát thấy tính mùa vụ của cổ phiếu

```
df_log = np.log(df)
moving_avg = df_log.rolling(12).mean()
df_log_detrend = df_log - moving_avg
df_log_detrend.dropna(inplace=True)
test_stationarity(df_log_detrend)
```

Cần thực hiện điều trên để phân tích dữ liệu sâu hơn. Đồng thời loại bỏ tính xu hướng để dự đoán.

Chạy autoArima để có tham số p, d, q tối ưu nhất

Tìm p, d, q tối ưu nhất trong giá trị [0 ; 10]

```
model_autoARIMA = auto_arima(
    train_data,
    start_p=0, start_q=0, start_d=0,
    max_p=10, max_q=10, max_d=5
    test='adf',
    m=1
```

```

seasonal=False,
trace=True,
error_action='ignore',
suppress_warnings=True,
stepwise=False,
n_jobs=-1)
print(model_autoARIMA.summary())
model_autoARIMA.plot_diagnostics(figsize=(15,8))
plt.show()
# Xây dựng mô hình
model_Arima = ARIMA(train_data, order=(5,0,0))
# Loại bỏ tham số disp=-1.
fitted = model_Arima.fit()
print(fitted.summary())
# Dự báo đã được sửa
forecast_result = fitted.get_forecast(steps=len(df_log_detrend) -
int(len(df_log_detrend)*0.8)) # số bước dự báo là 321
forecast_values = forecast_result.predicted_mean # lấy ra mảng numpy chứa
các giá trị dự báo
conf = forecast_result.conf_int(alpha=0.05) # khoảng tin cậy 95%

```

Tiến hành xây dựng mô hình

Loại bỏ tham số disp = -1.

```

# Calculate RMSE
rmse = math.sqrt(mean_squared_error(test_data, fc_series))
print(f'RMSE: {rmse}')

```

Tiến hành tính RMSE

```

# Lấy các cột khoảng tin cậy
conf_lower = conf.iloc[:, 0]
conf_upper = conf.iloc[:, 1]
# Đảm bảo giá trị dự báo là một mảng numpy.
fc = forecast_values.to_numpy()
# Tạo series pandas

```

```

fc_series = pd.Series(fc, index=test_data.index)
#thay đổi `conf[:,0]` và `conf[:,1]` để sử dụng iloc
lower_series = pd.Series(conf_lower, index=test_data.index)
upper_series = pd.Series(conf_upper, index=test_data.index)
# Vẽ đồ thị
plt.figure(figsize=(10,5), dpi=100)
plt.plot(test_data, color = 'blue', label='Actual Stock Price')
plt.plot(fc_series, color = 'orange',label='Predicted Stock Price')
plt.fill_between(lower_series.index, lower_series, upper_series,
                 color='k', alpha=.10)
plt.title( 'Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend(loc='upper left', fontsize=8)
plt.show()

```

Tiến hành dự báo và trực quan hoá kết quả

2. Thực hiện thuật toán

a. BNB-USD



Hình 11.1: Trung bình trượt và độ lệch chuẩn của BNB-USD

Results of Dickey-Fuller test

Test Statistics -0.997337

p-value 0.754234

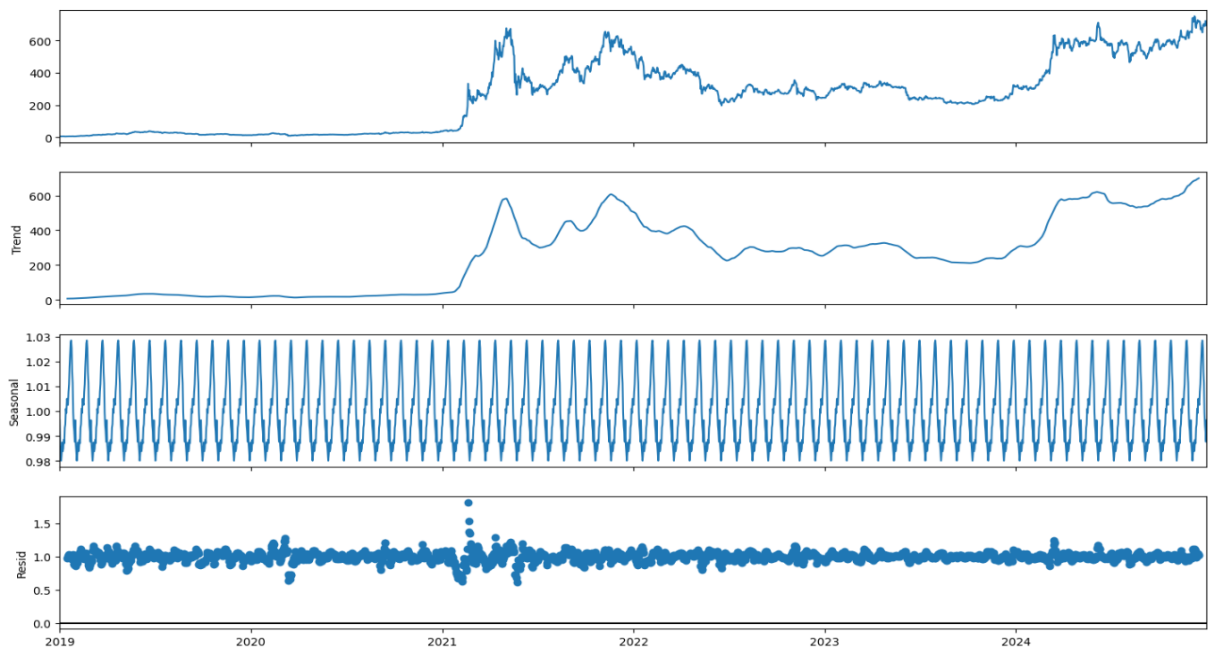
No. of lags used 25.000000

Number of observations used 2165.000000

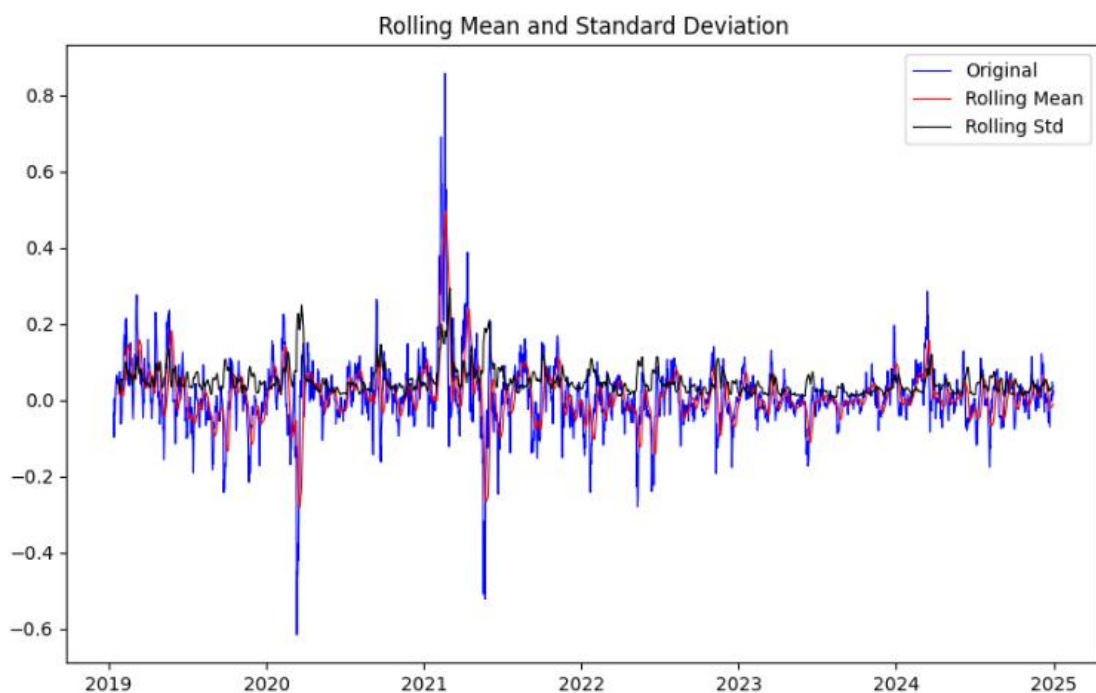
critical value (1%) -3.433374

critical value (5%) -2.862876

Ta có $p\text{-value} = 0.75 > 0.05 \rightarrow$ Giá BNB-USD không có tính dừng.



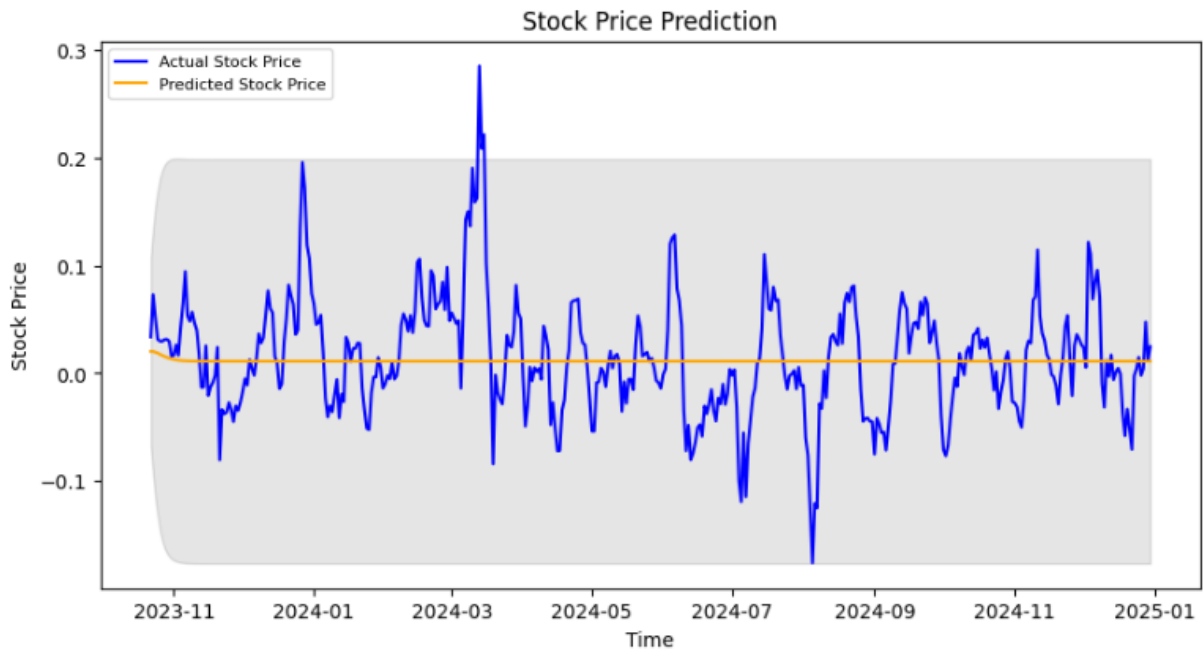
Hình 12.2: Phân tách chuỗi giá BNB-USD



Giá BNB-USD sau khi đã chuẩn hoá

Ta có $p\text{-value} < 0.05 \rightarrow$ Dữ liệu có tính dừng

Best model: ARIMA(5,0,0)(0,0,0)[0]



Test RMSE: 0.05377753431883073

Test RMSE cao, cho thấy thuật toán không đạt hiệu quả cao, đồng thời trên đồ thị cũng cho thấy thuật toán ARIMA không thể dự đoán giá cổ phiếu trong giai đoạn dài được.

→ Cần phải giảm tập kiểm tra

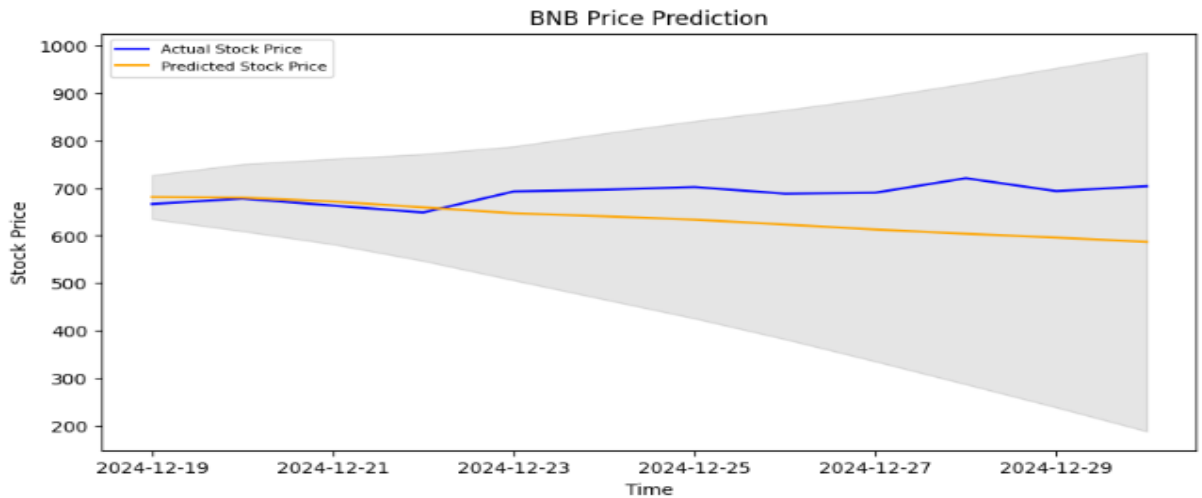
```
df = yf.download('BNB-USD', start='2024-11-01', end='2024-12-31')['Close']
#split data into train and training set
train_data, test_data = df[3:int(len(df)*0.8)], df[int(len(df)*0.8):]
plt.figure(figsize=(10,6))
plt.grid(True)
plt.xlabel('Dates')
plt.ylabel('Closing Prices')
plt.plot(df, 'green', label='Train data')
plt.plot(test_data, 'blue', label='Test data')
plt.legend()
```

Chuyển giá của BNB-USD từ ngày 01/11/2024 đến ngày 31/12/2024.

Tiến hành tìm model tối ưu

Xây dựng mô hình và dự đoán

Kết quả:



Kết quả thuật toán ARIMA của BNB-USD

Test RMSE: 69.33405765760423

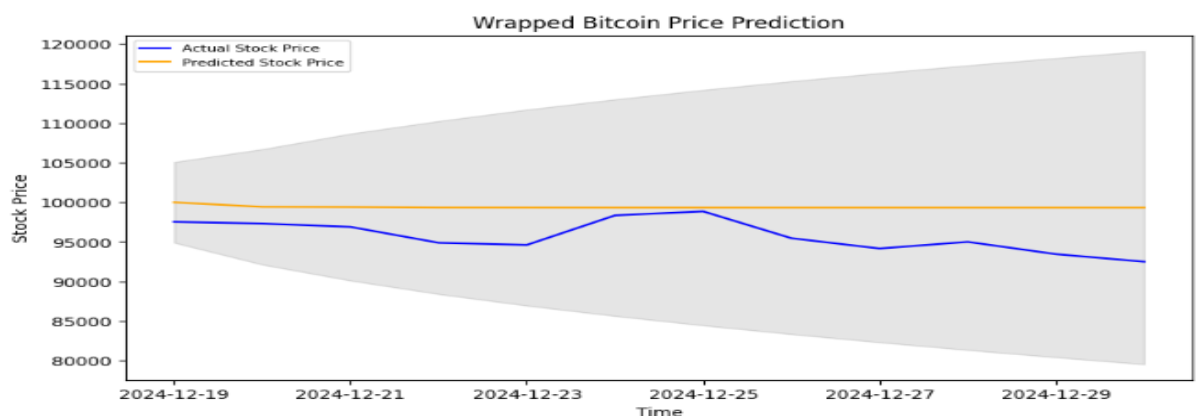
➔ Thuật toán đã đánh giá chính xác xu hướng giá đồng BNB, tuy rằng có sự chênh lệch. Đồng thời RMSE cũng có giá trị chấp nhận được (Khoảng 69) giúp ta có cái nhìn rõ ràng hơn về xu hướng.

b. WBTC

Tương tự như vậy, thực hiện trong giai đoạn 01/11/2024 đến 31/12/2024

Model tối ưu: (2; 1; 0)

Test RMSE : 4097.550358602954

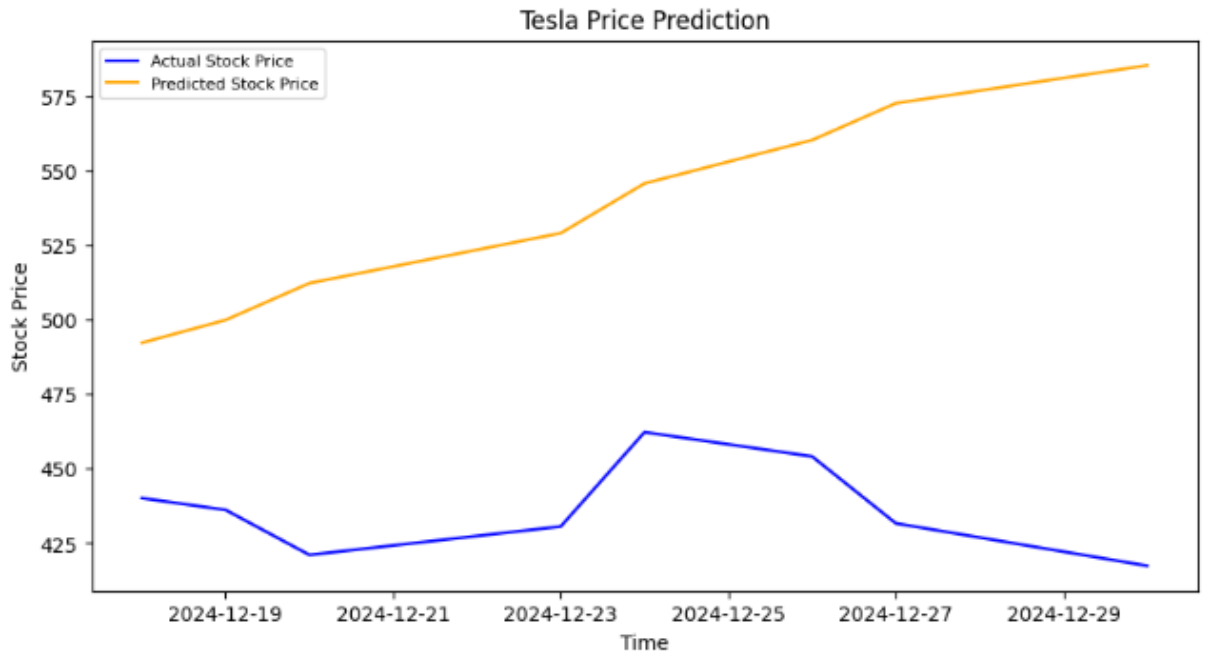


➔ Dự đoán chính xác xu hướng giá WBTC, tuy nhiên thuật toán có vẻ đang đánh giá giá trị của WBTC cao hơn so với thực tiễn.

c. TSLA

Model tối ưu : (4; 2; 1)

Test RMSE: 106.77495836292866



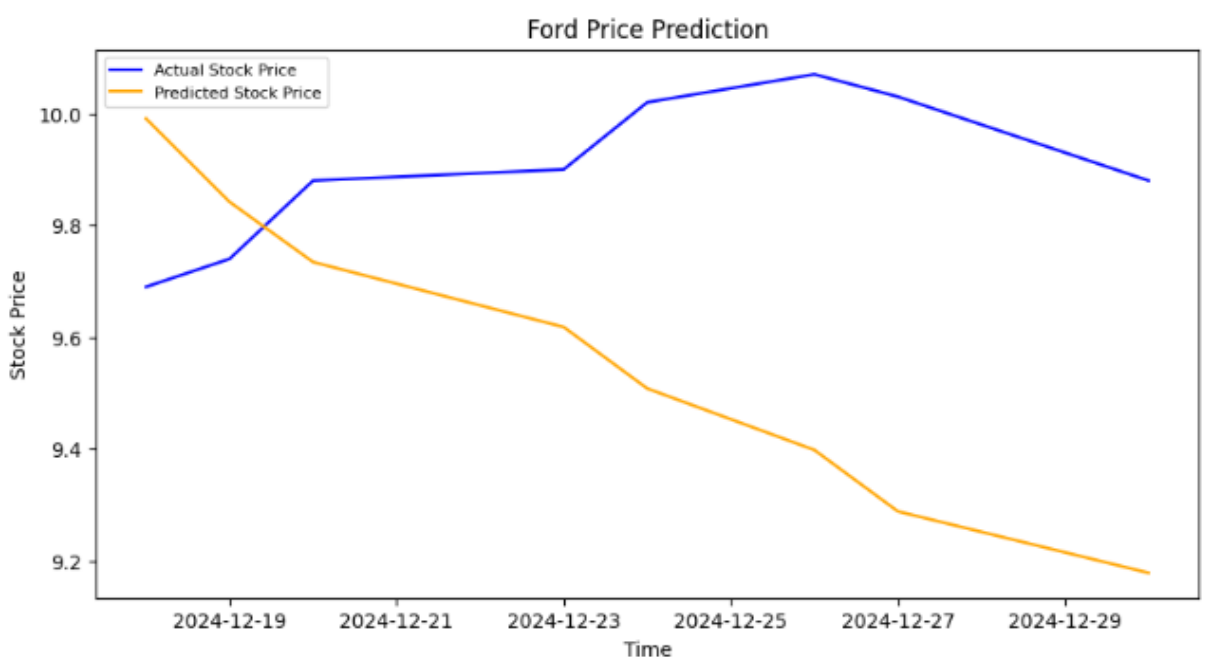
Kết quả thuật toán ARMA của TSLA

➔ Mô hình dự đoán sai hoàn toàn xu hướng. Kết quả của RMSE cũng cho thấy con số lớn (106.77495836292866) cho thấy thuật toán ARIMA không hoạt động hiệu quả với mã cổ phiếu TSLA

d. F

Model tối ưu (0; 2; 5)

Test RMSE: 0.49440811626815073



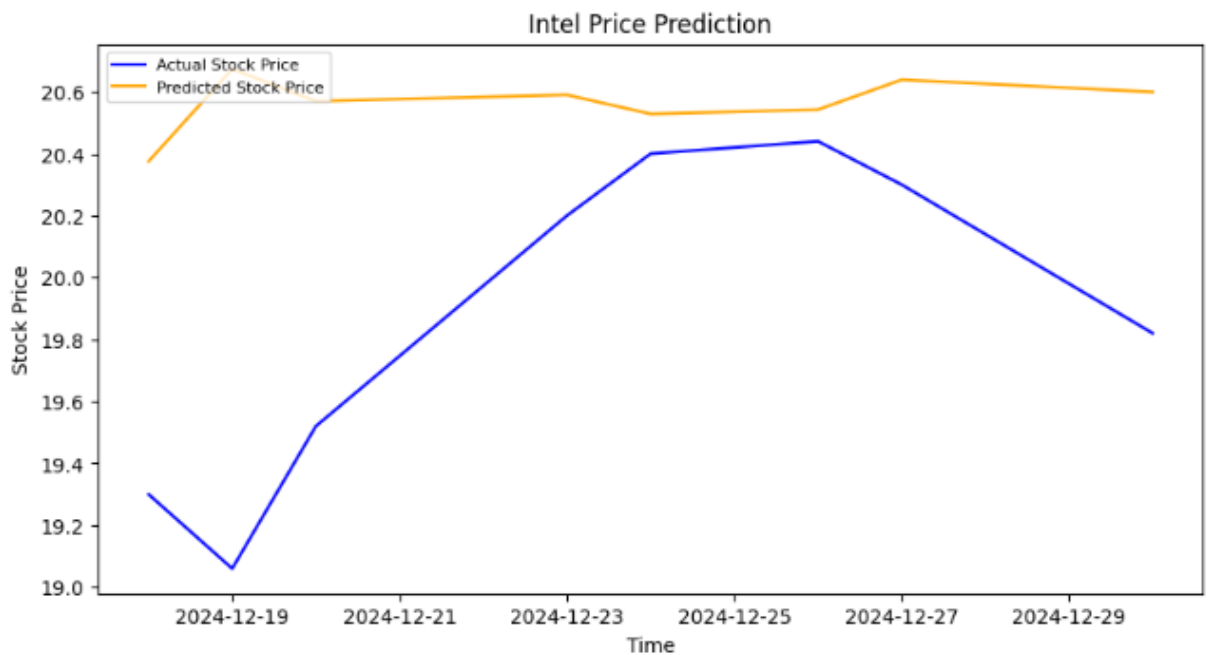
Kết quả thuật toán ARIMA của cổ phiếu F

➔ RMSE của cổ phiếu cao đối với giá của cổ phiếu F (0.5). Đồng thời trên đồ thị, thuật toán chỉ dự đoán chính xác từ giai đoạn cuối tháng 12.

e. INTC

Model tối ưu: (4; 2; 0)

Test RMSE: 0.8486191457947951



Kết quả thuật toán ARIMA với cổ phiếu INTC

➔ RMSE của cổ phiếu ở mức chấp nhận được (0.84). Thế nhưng trên đồ thị ta có thể thấy mô hình đang đánh giá giá trị cổ phiếu cao hơn thực tế, đồng thời đường dự báo cũng không bám sát so với thực tiễn.

VI. Đánh giá những thuật toán đã áp dụng

a. BNB – USD

Ta có Train RMSE của thuật toán SimpleRNN thấp hơn LSTM ($22.19 < 31.52$) điều đó cho thấy thuật toán SimpleRNN hoạt động tốt với ít sai số hơn.

Trên đồ thị trực quan, ta có thể thấy thuật toán SimpleRNN sẽ bám sát hơn so với thuật toán LSTM nếu như giá đồng BNB không bị biến động quá lớn.

Nếu như trong dự đoán ngắn hạn (Từ 1 đến 2 tháng), thuật toán Arima đem lại hiệu quả tốt hơn khi cho thấy đường xu hướng rõ ràng, thì trong dài hạn các thuật toán như SimpleRNN và LSTM đem lại hiệu quả vượt trội hoàn toàn.

➔ Nên sử dụng linh hoạt cho từng mục đích dự đoán.

b. WBTC-USD

Ta có Train RMSE của thuật toán SimpleRNN cao hơn LSTM ($3739.58 > 3563.01$).

Điều đó cho thấy thuật toán LSTM hoạt động tốt với ít sai số hơn.

Trên đồ thị trực quan, ta có thể thấy thuật toán SimpleRNN sẽ bám sát hơn so với thuật toán LSTM nếu như giá đồng BNB không bị biến động quá lớn. Song ở những tháng cuối năm giá WBTC có sự biến động mạnh, khiến cho thuật toán SimpleRNN không còn hoạt động hiệu quả mà có sai số lớn. Trong khi đó thuật toán LSTM dự đoán ổn định trong cả lúc giá biến động mạnh hay nhẹ.

Nếu như trong dự đoán ngắn hạn (Từ 1 đến 2 tháng), thuật toán Arima đem lại hiệu quả tốt hơn khi cho thấy đường xu hướng rõ ràng, thì trong dài hạn các thuật toán như SimpleRNN và LSTM đem lại hiệu quả vượt trội hoàn toàn.

➔ Nên sử dụng LSTM và SimpleRNN cho dự đoán dài, nên sử dụng thuật toán ARIMA cho dự đoán ngắn hạn.

c. TSLA

Ta có Train RMSE của thuật toán SimpleRNN thấp hơn LSTM ($15.09 < 17.59$).

Điều đó cho thấy thuật toán SimpleRNN hoạt động tốt với ít sai số hơn.

Trên đồ thị trực quan, ta có thể thấy thuật toán SimpleRNN sẽ bám sát hơn so với thuật toán LSTM nếu như giá đồng BNB không bị biến động quá lớn. Trong khi đó thuật toán LSTM sẽ dự đoán chính xác hơn đường xu hướng của cổ phiếu.

Nếu như trong dự đoán ngắn hạn (Từ 1 đến 2 tháng), thuật toán Arima đem lại hiệu quả tốt hơn khi cho thấy đường xu hướng rõ ràng, thì trong dài hạn các thuật toán như SimpleRNN và LSTM đem lại hiệu quả vượt trội hoàn toàn.

➔ Nên sử dụng LSTM và SimpleRNN cho dự đoán dài, nên sử dụng thuật toán ARIMA cho dự đoán ngắn hạn.

d. F

Ta có Train RMSE của thuật toán SimpleRNN thấp hơn LSTM ($0.43 < 0.63$). Điều đó cho thấy thuật toán SimpleRNN hoạt động tốt với ít sai số hơn.

Trên đồ thị trực quan, ta có thể thấy thuật toán SimpleRNN sẽ bám sát hơn so với thuật toán LSTM nếu như giá đồng BNB không bị biến động quá lớn. Trong khi đó thuật toán LSTM sẽ dự đoán chính xác hơn đường xu hướng của cổ phiếu.

Nếu như trong dự đoán ngắn hạn (Từ 1 đến 2 tháng), thuật toán Arima đem lại hiệu quả tốt hơn khi cho thấy đường xu hướng rõ ràng, thì trong dài hạn các thuật toán như SimpleRNN và LSTM đem lại hiệu quả vượt trội hoàn toàn.

➔ Nên sử dụng LSTM và SimpleRNN cho dự đoán dài, nên sử dụng thuật toán ARIMA cho dự đoán ngắn hạn.

e. INTC

Ta có Train RMSE của thuật toán SimpleRNN thấp hơn LSTM ($1.2 < 2.46$). Điều đó cho thấy thuật toán SimpleRNN hoạt động tốt với ít sai số hơn.

Trên đồ thị trực quan, ta có thể thấy thuật toán SimpleRNN sẽ bám sát hơn so với thuật toán LSTM nếu như giá đồng BNB không bị biến động quá lớn. Trong khi đó thuật toán LSTM sẽ dự đoán chính xác hơn đường xu hướng của cổ phiếu.

Nếu như trong dự đoán ngắn hạn (Từ 1 đến 2 tháng), thuật toán Arima đem lại hiệu quả tốt hơn khi cho thấy đường xu hướng rõ ràng, thì trong dài hạn các thuật toán như SimpleRNN và LSTM đem lại hiệu quả vượt trội hoàn toàn.

➔ Nên sử dụng LSTM và SimpleRNN cho dự đoán dài, nên sử dụng thuật toán ARIMA cho dự đoán ngắn hạn.

B. Thuật toán CNN để phân loại ảnh

I. Giới thiệu bộ dữ liệu

Bộ dữ liệu ‘Human_action_detection’ là bộ dữ liệu liên quan đến những hoạt động thường nhật của con người. Bộ dữ liệu bao gồm 7200 hình ảnh với định dạng jpeg, được phân thành 6 lớp đại diện cho 6 hoạt động của con người. 6 lớp bao gồm :

STT	Tên lớp	Chú thích
1	Cycling	Đạp xe
2	Dancing	Nhảy múa
3	Eating	Ăn
4	Fighting	Đánh nhau
5	Listening_to_music	Nghe nhạc
6	Running	Chạy bộ

Trong thời đại cách mạng công nghiệp 4.0, trí tuệ nhân tạo (AI) và học sâu (Deep Learning) đã trở thành những công cụ mạnh mẽ, hỗ trợ giải quyết nhiều bài toán phức tạp trong đời sống và công nghệ. Một trong những lĩnh vực ứng dụng đáng chú ý là phân loại hoạt động con người (Human Action Classification), đóng vai trò quan trọng trong các hệ thống giám sát, chăm sóc sức khỏe, thể thao và thực tế ảo.

Thuật toán học sâu đã chứng minh khả năng vượt trội trong việc trích xuất và phân tích đặc trưng từ dữ liệu video và hình ảnh. Đặc biệt, các mô hình mạng nơ-ron tích chập (Convolutional Neural Networks - CNN) như MobileNet, ResNet và AlexNet đã được áp dụng thành công trong nhiều bài toán thị giác máy tính. Với các kiến trúc tối ưu hóa và khả năng học đặc trưng ưu việt, ba mô hình này đã trở thành nền tảng trong nghiên cứu và ứng dụng.

Trong đề tài này, chúng tôi tập trung nghiên cứu và so sánh hiệu quả của các thuật toán MobileNet, ResNet và AlexNet trong việc phân loại hoạt động con người. Mục tiêu là đánh giá khả năng nhận diện và phân loại chính xác các hoạt động dựa trên dữ liệu đầu vào, đồng thời tìm ra mô hình phù hợp nhất cho từng trường hợp ứng dụng cụ thể. Kết quả nghiên cứu không chỉ góp phần nâng cao hiệu quả của các hệ thống nhận diện, mà còn mở ra hướng phát triển các ứng dụng thông minh phục vụ đời sống con người như: Giám sát an ninh, thể thao, giao thông thông minh...

II. Tiền xử lí dữ liệu

1. Nhập thư viện

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import backend as K
from tensorflow.keras.layers import Dense, Activation,Dropout,Conv2D,
MaxPooling2D,BatchNormalization, Flatten, Add, AveragePooling2D, Input,
ZeroPadding2D
from tensorflow.keras.layers import Activation, BatchNormalization, Add,
Reshape, DepthwiseConv2D, GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import categorical_crossentropy
from tensorflow.keras import regularizers
from tensorflow.keras.initializers import glorot_uniform
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model, load_model, Sequential
import numpy as np
import pandas as pd
import shutil
import time
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from matplotlib.pyplot import imshow
import os
import seaborn as sns
sns.set_style('darkgrid')
from PIL import Image
from sklearn.metrics import confusion_matrix, classification_report
from tensorflow.keras.metrics import Recall, Precision, AUC,
RootMeanSquaredError
from sklearn.metrics import f1_score
```

2. Tiến hành giảm số lớp

Dữ liệu gốc bao gồm 15 lớp, song để đơn giản và giảm thời gian thực hiện thuật toán, người thực hiện tiến hành giảm xuống chỉ còn 6 lớp.

```
class_names = ['cycling', 'dancing', 'eating', 'fighting', 'listening_to_music',
               'running']
class_names_label = {class_name:i for i, class_name in enumerate(class_names)}

nb_classes = len(class_names)
IMAGE_SIZE = (224, 224)
class_names_label
```

Chỉnh lại kích cỡ ảnh thành 224 x 224 đồng đều.

3. Đếm số ảnh của mỗi lớp

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("emirhanai/human-action-detection-artificial-
intelligence")

print("Path to dataset files:", path)
(train_images, train_labels), (test_images, test_labels) = load_data(datasets)
```

Tiến hành lấy ảnh từ Kaggle

Tải số ảnh đó để lưu trữ và tiến hành thuật toán

```
import numpy as np
from sklearn.model_selection import train_test_split

def full_sample(images, labels):
    # Trả về toàn bộ dữ liệu mà không cần lấy mẫu
    return np.array(images), np.array(labels)

# Sử dụng full sample cho dữ liệu huấn luyện và kiểm tra
train_images_full, train_labels_full = full_sample(train_images, train_labels)
```

```
test_images_full, test_labels_full = full_sample(test_images, test_labels)

print("Full train images shape:", train_images_full.shape)
print("Full test images shape:", test_images_full.shape)
```

Đếm số ảnh của tập huấn luyện và tập kiểm tra

- Tập huấn luyện bao gồm 6000 ảnh chia đều cho 6 lớp
- Tập kiểm tra bao gồm 1200 ảnh chia đều cho 6 lớp

Vẽ biểu đồ cột để đếm số ảnh của mỗi lớp.



➔ Mỗi lớp gồm có 1000 ảnh trong tập huấn luyện và 200 ảnh trong tập kiểm tra ➔ Việc chia đều giúp cho việc học của máy trở nên công bằng hơn, giúp đạt kết quả tốt hơn.

4. Chuẩn hoá dữ liệu với phương pháp Min_Max_Normalization

```
# Chuẩn hóa (normalize) các giá trị pixel xuống [0, 1]
train_images_full = train_images_full / 255.0
test_images_full = test_images_full / 255.0
```

Chuẩn hoá cỡ ảnh về giá trị [0 ;1]

5. Hiện ảnh ngẫu nhiên với mỗi lớp hoạt động


```
def display_examples(class_names, images, labels):
    fig = plt.figure(figsize=(10,10))
    fig.suptitle("Some examples of images of the dataset", fontsize=16)
    for i in range(25):
        plt.subplot(5,5,i+1)
        plt.xticks([]) # Ẩn các nhãn trục x
        plt.yticks([]) # Ẩn các nhãn trục y
        plt.grid(False) # Tắt lưới
        plt.imshow(images[i], cmap=plt.cm.binary) # Hiển thị ảnh
        plt.xlabel(class_names[labels[i]]) # Nhãn lớp của hình ảnh
    plt.show()
display_examples(class_names, train_images_full, train_labels_full)
```

Tiến hành hiện 25 bức ảnh ngẫu nhiên từ các lớp của tập huấn luyện

Some examples of images of the dataset



III. Thực hiện thuật toán Alexnet với bộ dữ liệu

1. Tạo ra mạng Alexnet để tiến hành học sâu với bộ dữ liệu

Layer	Type	Map	Size	Kernel Size	Stride	Padding	Activation
Out	Output	6	-	-	-	-	Softmax
D2	-	8192	-	-	-	-	ReLU
D1	-	8192	-	-	-	-	ReLU
C5	Convolution	512	13 x 13	3 x 3	2	Same	ReLU
C4	Convolution	512	13 x 13	3 x 3	2	Same	ReLU
C3	Convolution	512	13 x 13	3 x 3	2	Same	ReLU
P2	Max Pooling	96	26 x 26	3 x 3	2	Same	
C2	Convolution	256	26 x 26	5 x 5	2	Same	ReLU
P1	Max Pooling	96	26 x 26	3 x 3	2	Same	
C1	Convolution	96	54 x 54	11 x 11	4	Same	ReLU
In	Input	3 (RGB)	224 x 224	-	-	-	-

2. Tiến hành thuật toán phân loại hoạt động với mô hình Alexnet

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', linestyle='--')
plt.title('Training and Validation Accuracy vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
```

```

plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(12, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss', linestyle='--')
plt.title('Training and Validation Loss vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
lr_history = history.history.get('lr', []) #
if lr_history:
    plt.figure(figsize=(12, 6))
    plt.plot(lr_history, label='Learning Rate')
    plt.title('Learning Rate vs Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('Learning Rate')
    plt.legend()
    plt.grid(True)
    plt.show()

```

Sử dụng Loss để kiểm tra mức độ thất thoát của thuật toán và Accuracy để đánh giá kết quả thuật toán

```

# Compile mô hình với learning rate scheduler và optimizer mới
lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.001, # Learning rate ban đầu
    decay_steps=10000, # Số bước giảm learning rate
    decay_rate=0.95, # Tỷ lệ giảm
    staircase=True)

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),

```

```

        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])
# Xem tổng quan mô hình
model.summary()
# Huấn luyện mô hình
history = model.fit(train_images_full, train_labels_full, batch_size=128,
epochs=15)

```

Cho Batch_size thành 128 và tiến hành thuật toán học sâu với 15 lần.

Chạy mô hình với learning rate và optimizer đã được tối ưu.

```

y_pred = model.predict(test_images_full)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(test_labels_full, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.arange(6),
yticklabels=np.arange(6), cbar=False)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()

```

Vẽ ma trận nhầm lẫn để kiểm tra kết quả dự đoán của mỗi lớp.

3. Kết quả của thuật toán

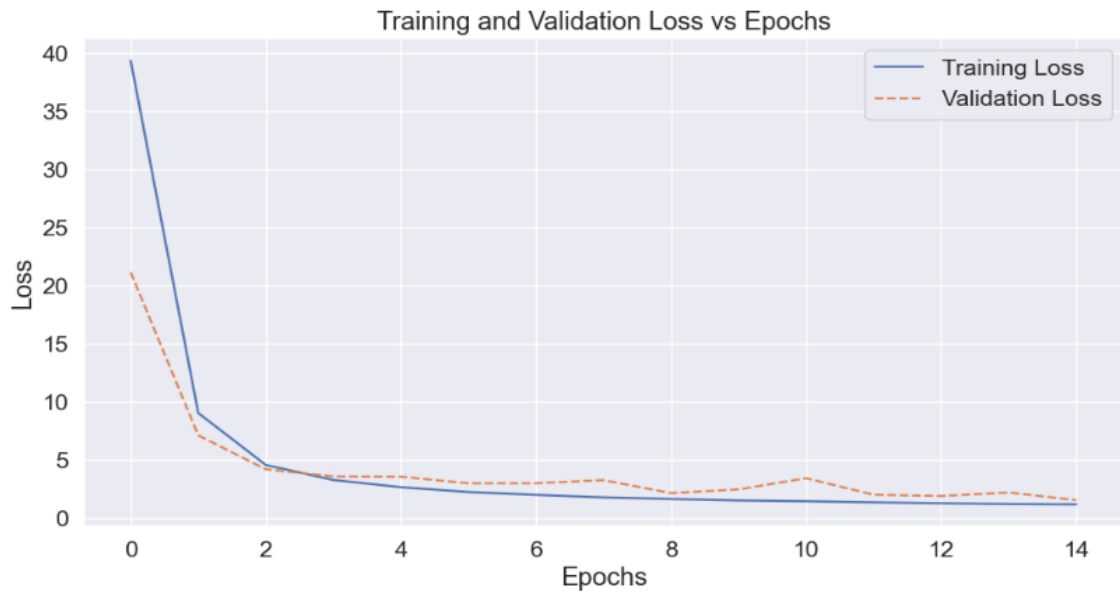
Epoch 15/15

38/38 ————— 186s 5s/step - accuracy:
0.6376 - loss: 1.1825 - val_accuracy: 0.5183 - val_loss: 1.5455

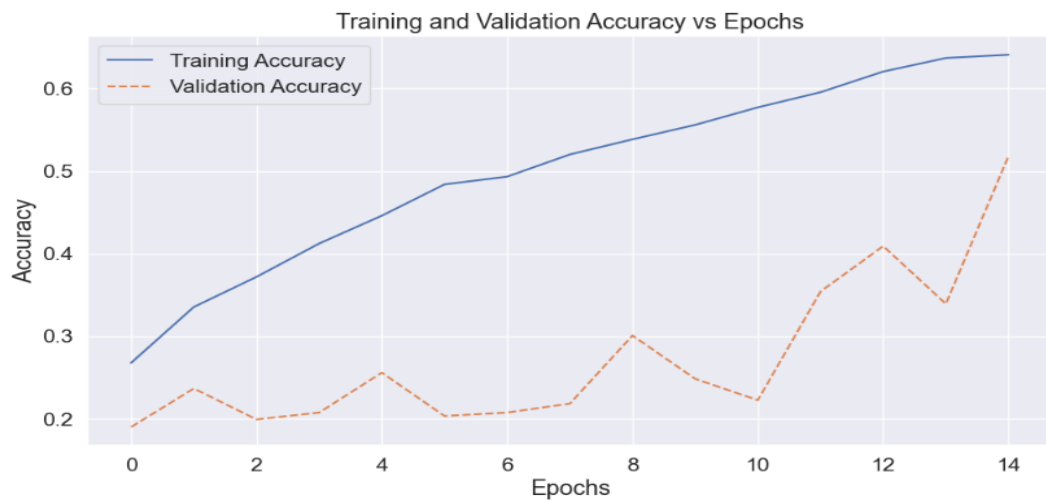
Kết quả phân loại ở tập huấn luyện chỉ là 63.76% và ở tập kiểm tra là 51.83%.

➔ Mô hình phân loại kém hiệu quả

Thất thoát ở cả tập huấn luyện và tập kiểm tra đều rất khả quan : Giảm mạnh từ 40 (tập kiểm thử) và 20 (tập kiểm tra) xuống dưới 1 và tiến rất sát về 0.



Thất thoát của thuật toán Alexnet



Độ chính xác của thuật toán Alexnet

Độ chính xác ở tập huấn luyện tăng đều từ 0.28 lên 0.63 trong suốt 15 lần học. Trong khi đó với tập kiểm tra kết quả tăng mạnh rồi sau đó có sự sụt giảm như đồ thị hình sin và cuối cùng có kết quả là 0.52.

Ma trận nhầm lẫn (Confusion Matrix):

Confusion Matrix

True Label \ Predicted Label	0	1	2	3	4	5
0	128	12	0	8	3	49
1	14	128	1	20	10	27
2	23	38	74	11	35	19
3	13	61	2	97	4	23
4	16	41	0	8	98	37
5	42	35	3	11	6	103

Nhận xét:

- Nhìn vào confusion matrix, ta thấy:
- Các nhãn khác nhau (từ 0 đến 5) đều có sự phân loại sai đáng kể.
- Có sự nhầm lẫn cao giữa các nhãn lân cận, ví dụ:
- Nhiều trường hợp của nhãn 2 bị dự đoán thành nhãn 1 (38) và nhãn 4 (35).
- Nhãn 5 cũng bị nhầm lẫn nhiều với nhãn 0 (42) và nhãn 1 (35).
- Một số nhãn như nhãn 0 và 1 có hiệu suất tốt hơn so với các nhãn khác, với nhiều giá trị dự đoán đúng (128) so với số sai.

Nguyên nhân:

- Bộ dữ liệu tương đối khó cho việc học máy, việc các ảnh giữa các lớp có sự tương đồng khá lớn, dẫn đến lúc học gặp khó khăn.
- Số lượng ảnh không quá lớn, có thể bổ sung từ 1000 đến 2000 ở mỗi lớp để tiến hành học sâu
- Thuật toán Alexnet không hoạt động hiệu quả với bộ dữ liệu.

IV. Sử dụng thuật toán Mobilenet để phân loại

1. Tạo ra mạng Mobilenet

```

mobilenet = MobileNet(weights='imagenet', include_top=False)
# freeze the layers
for layer in mobilenet.layers:
    layer.trainable = False
# create a single input layer
input_layer = Input(shape=(224,224,3))
# connect the input layer to both models
x1 = mobilenet(input_layer)
# concatenate the outputs from both models
concatenated = Concatenate()([x1])
# add a few dense layers for the final prediction
flat = Flatten()(concatenated)
dense = Dense(units=1024, activation='relu')(flat)
dense = Dense(512, activation='relu')(dense)
dense = Dense(units=256, activation='relu')(dense)
predictions = Dense(num_classes, activation='softmax')(dense)
# compile the model
# this is the model we will train
model = Model(inputs=input_layer, outputs=predictions)

```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
mobilenet_1.00_224 (Functional)	(None, 7, 7, 1024)	3,228,864
concatenate (Concatenate)	(None, 7, 7, 1024)	0
flatten (Flatten)	(None, 50176)	0
dense (Dense)	(None, 1024)	51,381,248
dense_1 (Dense)	(None, 512)	524,800
dense_2 (Dense)	(None, 256)	131,328
dense_3 (Dense)	(None, 6)	1,542

2. Tiến hành thuật toán phân loại hoạt động với mô hình Mobilenet

InputLayer: Dữ liệu đầu vào có kích thước (224, 224, 3).

MobileNet: Kích thước đầu ra (None, 7, 7, 1024).

Flatten: Chuyển đổi tensor 3D (7, 7, 1024) thành vector 1D (50176).

Các tầng Dense giảm dần kích thước đầu ra: $1024 \rightarrow 512 \rightarrow 256 \rightarrow 5$.

Trainable params: Các tham số được tối ưu trong quá trình huấn luyện (hầu hết các tham số Dense).

Non-trainable params: Tham số không được cập nhật, thường là tham số từ mô hình pre-trained (MobileNet).

```
class LRA(keras.callbacks.Callback):
    reset = False
    count = 0
    stop_count = 0
    tepochs = 0
    best_weights = None
    def __init__(self, model, patience, stop_patience, threshold, factor, dwell,
model_name, freeze, initial_epoch):
        super(LRA, self).__init__()
        self._model = model
        self.patience = patience
        self.stop_patience = stop_patience
        self.threshold = threshold
        self.factor = factor
        self.dwell = dwell
        self.lr = float(tf.keras.backend.get_value(model.optimizer.learning_rate))
        self.highest_tracc = 0.0
        self.lowest_vloss = np.inf
        self.initial_epoch = initial_epoch
        LRA.best_weights = model.get_weights()
        msgs = (f' Starting training using base model {model_name} with weights frozen
to imagenet weights initializing LRA callback'
                if freeze else
                f' Starting training using base model {model_name} training all layers ')
        print_in_color(msgs, (244, 252, 3), (55,65,80))
    def model(self):
        return self._model
```



```

def model(self, value):
    self._model = value

def on_epoch_begin(self, epoch, logs=None):
    self.now = time.time()

def on_epoch_end(self, epoch, logs=None):
    later = time.time()
    duration = later - self.now

    if epoch == self.initial_epoch or LRA.reset:
        LRA.reset = False

        msg = '{0:^8s}{1:^10s}{2:^9s}{3:^9s}{4:^9s}{5:^9s}{6:^9s}'.format(
            'Epoch', 'Loss', 'Acc', 'V_loss', 'V_acc', 'LR', 'Next LR'
        )
        print_in_color(msg, (244,252,3), (55,65,80))

    lr = float(tf.keras.backend.get_value(self.model.optimizer.learning_rate))
    current_lr = lr

    v_loss = logs.get('val_loss')
    acc = logs.get('accuracy')
    v_acc = logs.get('val_accuracy')
    loss = logs.get('loss')

    if acc < self.threshold:
        if acc > self.highest_tracc:
            self.highest_tracc = acc

            LRA.best_weights = self.model.get_weights()
            self.count = 0
            self.stop_count = 0

            if v_loss < self.lowest_vloss:
                self.lowest_vloss = v_loss

            self.lr = lr
        else:
            if self.count >= self.patience - 1:
                self.lr = lr * self.factor

                tf.keras.backend.set_value(self.model.optimizer.learning_rate, self.lr)

```

```

        self.count = 0
        self.stop_count += 1
        if self.dwell:
            self.model.set_weights(LRA.best_weights)
        else:
            self.count += 1
    else:
        if v_loss < self.lowest_vloss:
            self.lowest_vloss = v_loss
            LRA.best_weights = self.model.get_weights()
            self.count = 0
            self.stop_count = 0
            self.lr = lr
        else:
            if self.count >= self.patience - 1:
                self.lr = self.lr * self.factor
                self.stop_count += 1
                self.count = 0
                tf.keras.backend.set_value(self.model.optimizer.learning_rate, self.lr)
                if self.dwell:
                    self.model.set_weights(LRA.best_weights)
            else:
                self.count += 1

    # Print training progress
    msg = f'{str(epoch+1):^3s}/{str(LRA.tepochs):4s}'
    {loss:^9.3f}{acc:^9.3f}{v_loss:^9.3f}{v_acc:^9.3f}{current_lr:^9.5f}{self.lr:^9.5f}'
    print_in_color(msg, (0, 67, 54), (55,65,80))

    if self.stop_count > self.stop_patience - 1:
        msg = f'training has been halted at epoch {epoch + 1} after'
        {self.stop_patience} adjustments of learning rate with no improvement'

```

```

print_in_color(msg, (0,255,0), (55,65,80))

self.model.stop_training = True

def print_in_color(txt, fore_color, back_color):
    print(f'\033[38;2;{fore_color[0]};{fore_color[1]};{fore_color[2]}m\033[48;2;{back_color[0]};{back_color[1]};{back_color[2]}m{txt}\033[0m')

# Define freeze before using it
freeze = False

model = tf.keras.applications.MobileNet(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
model.trainable = not freeze

model = tf.keras.Sequential([
    model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(len(classes), activation='softmax')
])

model = tf.keras.applications.MobileNet(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
model.trainable = not freeze

model = tf.keras.Sequential([
    model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(len(classes), activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

epochs = 5

```

```

patience = 2
stop_patience = 2
threshold = 0.8
factor = 0.8
dwell = True
freeze = False

callbacks = [LRA(model=model,
                 patience=patience,
                 stop_patience=stop_patience,
                 threshold=threshold,
                 factor=factor,
                 dwell=dwell,
                 model_name='MobileNet',
                 freeze=freeze,
                 initial_epoch=0)]

LRA.tepochs = epochs
history = model.fit(x=train_gen,
                   epochs=epochs,
                   callbacks=callbacks,
                   verbose=1,
                   validation_data=valid_gen,
                   validation_steps=None,
                   shuffle=False,
                   initial_epoch=0)

```

Quy trình hoạt động của mô hình này như sau:

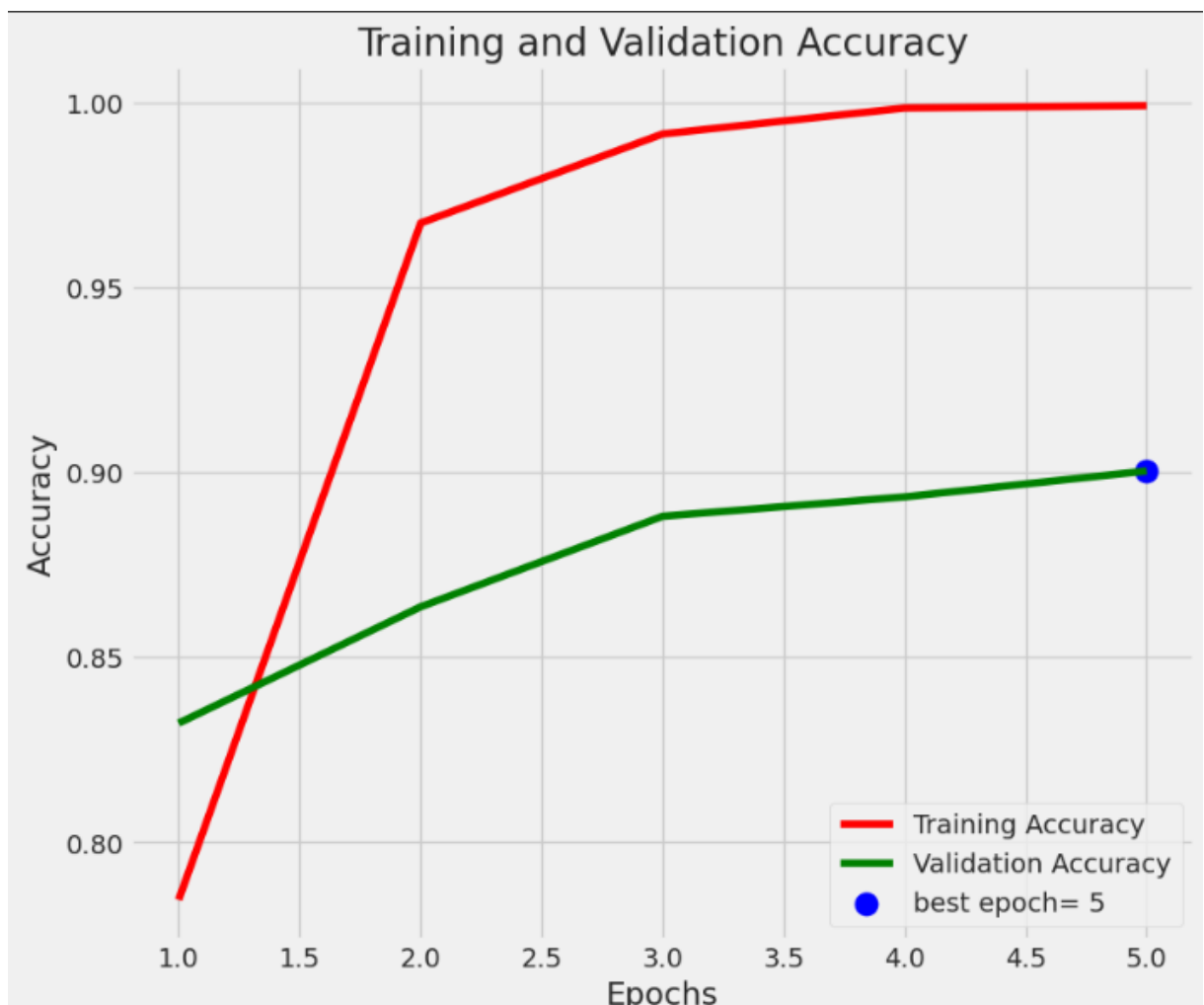
- Khởi tạo callback:
- Lưu trạng thái ban đầu của mô hình, learning rate, trọng số tốt nhất (best_weights).
- In thông báo bắt đầu huấn luyện (dựa trên việc đóng băng hay không các layer của mô hình).
- Trong mỗi epoch:

- Theo dõi loss và accuracy.
- Kiểm tra cải thiện: Nếu có, lưu lại trạng thái mô hình. Nếu không, giảm learning rate khi vượt ngưỡng patience và sử dụng trọng số tốt nhất nếu dwell=True.
- Dừng huấn luyện:
- Nếu giảm learning rate quá số lần quy định (stop_patience), callback dừng huấn luyện và in thông báo.

3. Kết quả thuật toán

Epoch 5/5

143/143 ————— 936s 7s/step - accuracy: 0.9995 - loss: 0.0158 - val_accuracy: 0.9003 - val_loss: 0.3117

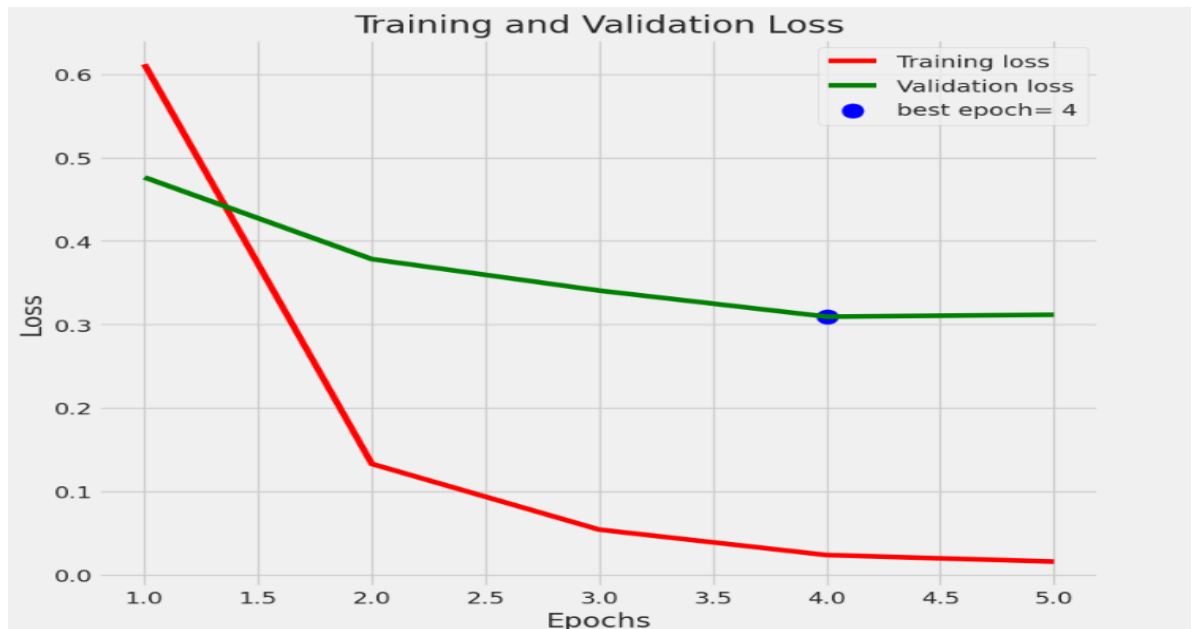


Độ chính xác của thuật toán Mobilenet

accuracy on the test set is 88.44 %

Nhận xét: Mô hình đạt hiệu suất rất cao, cả training và validation đều đạt trên 99%. Không có dấu hiệu overfitting nghiêm trọng khi cả loss và accuracy của training/validation khá sát nhau.

Những lần học đều có thất thoát thấp, cho thấy thuật toán đã sử dụng hiệu quả.



Ma trận nhầm lẫn:

	cycling	dancing	eating	fighting	listening_to_music	running
cycling	89	0	0	4	2	4
dancing	0	71	0	10	3	7
eating	1	0	98	2	1	0
fighting	1	7	1	66	1	3
listening_to_music	1	1	2	3	100	1
running	1	6	1	3	0	81

Nhận xét:

- Mô hình hoạt động rất hiệu quả với độ chính xác chủ yếu trên 90%, trừ tập dancing và fighting.
- Mô hình hoạt động rất tốt cho các lớp như "eating" và "listening to music".
- Mô hình thực sự hoạt động hiệu quả, nhưng cần xem xét để tránh trường hợp overfitting.

V. Ứng dụng thuật toán Resnet để phân loại hoạt động

1. Tạo ra mạng Resnet

```
# Tải ResNet50 với các trọng số pre-trained trên ImageNet
resnet50 = ResNet50(weights='imagenet', include_top=False)

# Đóng băng các lớp trong cả hai mô hình
for layer in resnet50.layers:
    layer.trainable = False

# Tạo một input layer duy nhất
input_layer = Input(shape=(224, 224, 3))

# Kết nối input layer với ResNet50 và Xception
x1 = resnet50(input_layer)

# Kết hợp (concatenate) đầu ra của hai mô hình
concatenated = Concatenate()([x1])

# Thêm các lớp Dense để thực hiện dự đoán
flat = Flatten()(concatenated)
dense = Dense(units=1024, activation='relu')(flat)
dense = Dense(units=512, activation='relu')(dense)
dense = Dense(units=256, activation='relu')(dense)
num_classes = 6 # Số lượng lớp (bạn có thể thay đổi)
predictions = Dense(num_classes, activation='softmax')(dense)
```

```
# Tạo mô hình
model = Model(inputs=input_layer, outputs=predictions)

# Biên dịch mô hình
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# In kiến trúc mô hình
model.summary()
```

Cấu trúc của mô hình Resnet:

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 1024)	23,587,712
concatenate (Concatenate)	(None, 7, 7, 1024)	0
flatten (Flatten)	(None, 100352)	0
dense (Dense)	(None, 1024)	102,761,472
dense_1 (Dense)	(None, 512)	524,800
dense_2 (Dense)	(None, 256)	131,328
dense_3 (Dense)	(None, 6)	1,542

2. Tiến hành thuật toán Resnet

InputLayer: Dữ liệu đầu vào có kích thước (224, 224, 3).

MobileNet: Kích thước đầu ra (None, 7, 7, 1024).

Flatten: Chuyển đổi tensor 3D (7, 7, 1024) thành vector 1D (50176).

Các tầng Dense giảm dần kích thước đầu ra: $1024 \rightarrow 512 \rightarrow 256 \rightarrow 5$.

Trainable params: Các tham số được tối ưu trong quá trình huấn luyện (hầu hết các tham số Dense).

Non-trainable params: Tham số không được cập nhật, thường là tham số từ mô hình pre-trained (MobileNet).

Lớp Dense_3 là đầu ra với số lớp là 6.

3. Kết quả thuật toán.

Sử dụng chung những thư viện của thuật toán Mobilenet.

Quy trình hoạt động của mô hình này như sau:

- Khởi tạo callback:
- Lưu trạng thái ban đầu của mô hình, learning rate, trọng số tốt nhất (best_weights).
- In thông báo bắt đầu huấn luyện (dựa trên việc đóng băng hay không các layer của mô hình).
- Trong mỗi epoch:
 - Theo dõi loss và accuracy.
 - Kiểm tra cải thiện: Nếu có, lưu lại trạng thái mô hình. Nếu không, giảm learning rate khi vượt ngưỡng patience và sử dụng trọng số tốt nhất nếu dwell=True.
 - Dừng huấn luyện:
 - Nếu giảm learning rate quá số lần quy định (stop_patience), callback dừng huấn luyện và in thông báo.

Epoch 5/5

143/143 ————— 809s 6s/step -accuracy:0.9955 -

loss: 0.0173 - val_accuracy: 0.7080 - val_loss: 1.1044



Thất thoát của mô hình Resnet

Tập kiểm tra có sai số khá lớn (2.5) so với tập huấn luyện (0.52)

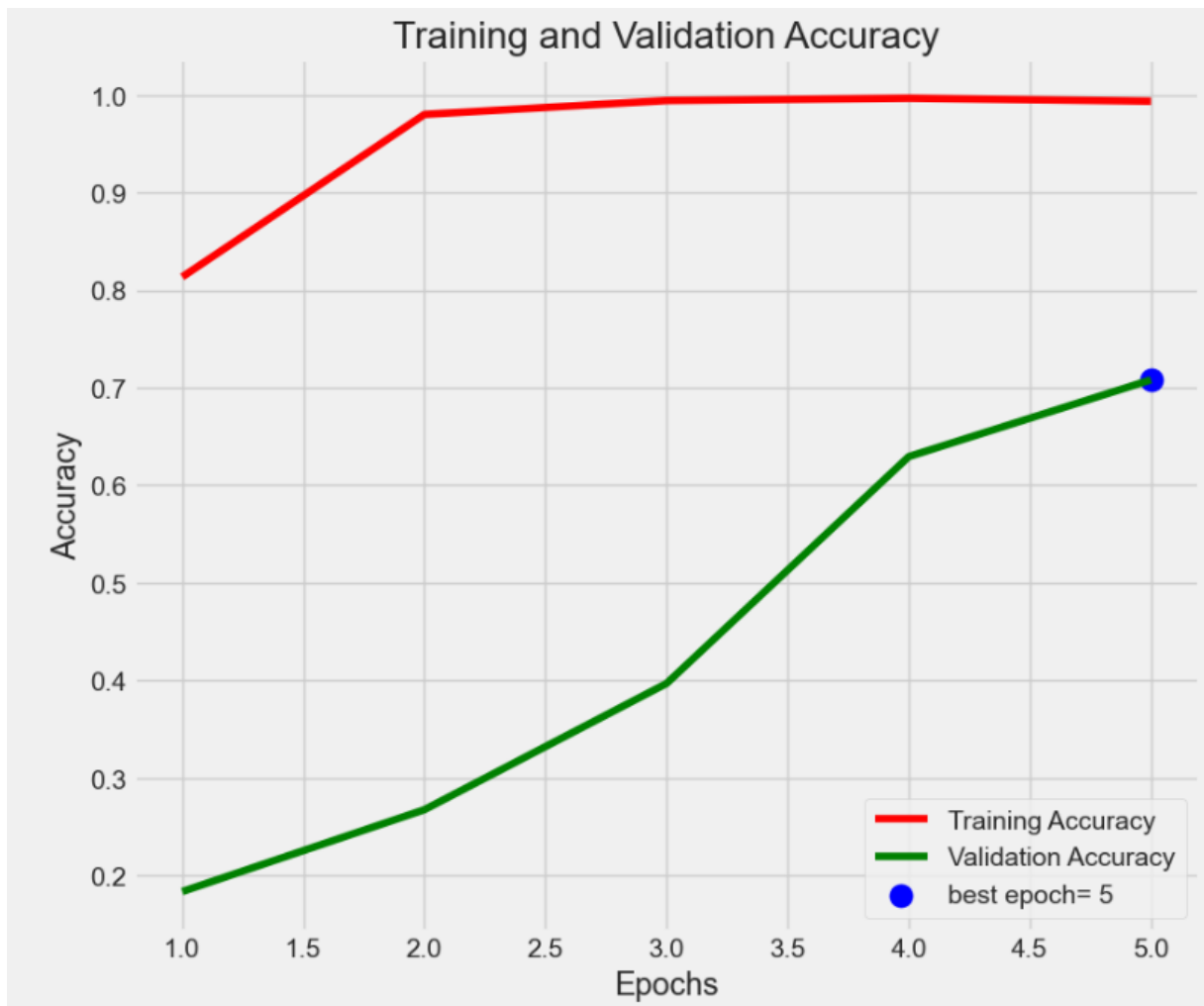
Trong 2 lần học đầu tiên, thất thoát tăng dần từ 2.4 lên 2.5. Sau đó giảm dần xuống gần về 1.

Ở tập huấn luyện, sai số giảm mạnh ở lần học thứ 2 khi giảm từ 0.52 xuống gần bằng 0. Sau đó ổn định ở mức rất thấp.

571/571 ————— 41s 71ms/step - accuracy:

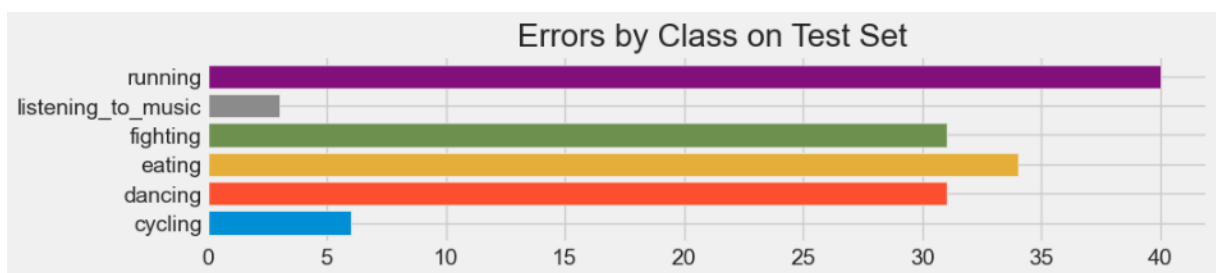
0.7666 - loss: 0.8044

accuracy on the test set is 74.61 %



Độ chính xác mô hình Resnet

Hai biểu đồ ở trên cho thấy mô hình được huấn luyện tốt, với Accuracy cao nhất đạt tới 99,95%. Tuy nhiên, khi sử dụng mô hình đã được huấn luyện để kiểm thử thì kết quả cho thấy mô hình đạt hiệu suất kém hơn (độ chính xác chỉ đạt vừa hơn 70%). Điều này cho thấy mô hình Resnet đang có nguy cơ bị Overfitting.



Mà trận nhầm lẫn:

Confusion Matrix

Actual	cycling	86	1	0	1	2	2
	dancing	2	77	0	17	10	2
	eating	0	6	45	7	21	0
	fighting	2	11	0	71	12	6
	listening_to_music	0	1	0	0	88	2
	running	6	15	0	13	6	59
		cycling	dancing	eating	fighting	_to_music	running

Nhận xét:

- Một số nhãn như "cycling" và "listening_to_music" được dự đoán khá chính xác.
- Nhãn "eating" cũng bị nhầm khá nhiều với "dancing" và "fighting".
- Nhãn "eating" là lớp bị phân loại sai sót nhiều nhất.
- Nhãn "running" cũng gặp rất nhiều sai sót.

VI. Đánh giá, so sánh kết quả phân loại của các thuật toán

Theo độ chính xác của tập kiểm tra:

- Mô hình Alexnet đem lại hiệu quả kém nhất (51.83%)
- Mô hình Resnet đem lại hiệu quả tốt hơn (74.61%)
- Mô hình Mobilenet đem lại hiệu quả tốt nhất (88.44%)

Đồng thời mô hình Mobilenet cũng xử lý với thời gian ngắn nhất (2s/step)

➔ Với bộ ảnh phân loại hoạt động của con người, người thực hiện kiến nghị ưu tiên mô hình Alexnet với tốc độ xử lý cao và độ chính xác tốt, tiếp đến là Resnet và cuối cùng là Alexnet.